

Abdul Wahid
EC-Council CND (Certified Network Defender)

Administrative Application Security – Whitelisting, Sandboxing & Web Filtering

Executive Summary

Administrative Application Security focuses on implementing and validating application-level security controls across Windows and Linux environments — specifically application whitelisting, sandboxing, and web application filtering. Across 5 labs, three core defense strategies were demonstrated: restricting which applications can execute (AppLocker, SRP), isolating untrusted code from the host system (Firejail, Sandboxie-Plus), and filtering malicious web requests at the server level (IIS Request Filtering).

All labs were executed hands-on across a multi-VM lab environment (Windows Server 2019, Windows 10, and Ubuntu 64-bit), with each control independently configured, tested, and — critically — verified through live attack simulation rather than assumed to work from configuration alone. Two labs required tool substitutions (Microsoft URLScan → IIS Request Filtering; Shade Sandbox → Sandboxie-Plus) due to the original tools being discontinued or paid; both substitutes achieve functionally identical security outcomes and are documented as such.

A notable strength of this module's execution was the troubleshooting encountered in Labs 1–2, where a real AppLocker/SRP enforcement conflict was diagnosed and resolved rather than glossed over — producing a more technically credible result than simply following steps.

Lab Summary

Lab	Objective	Tool Used	Result
1	Application whitelisting via domain-style policy	AppLocker (Local Security Policy)	Rules enforced; blocked app confirmed via live test
2	Alternative whitelisting mechanism	Software Restriction Policies	Rules enforced; block verified after isolating AppLocker interference
3	Sandboxing Untrusted Applications (Linux)	Firejail	Network + filesystem isolation proven; --private mode validated
4	Blocking web-based attacks (XSS) at the server	IIS Request Filtering *(substitute for URLScan)*	XSS payload blocked (HTTP 404) after filter applied
5	Safe execution of untrusted/malicious files	Sandboxie-Plus *(substitute for Shade Sandbox)*	Real malware executed safely in isolation; host system unaffected

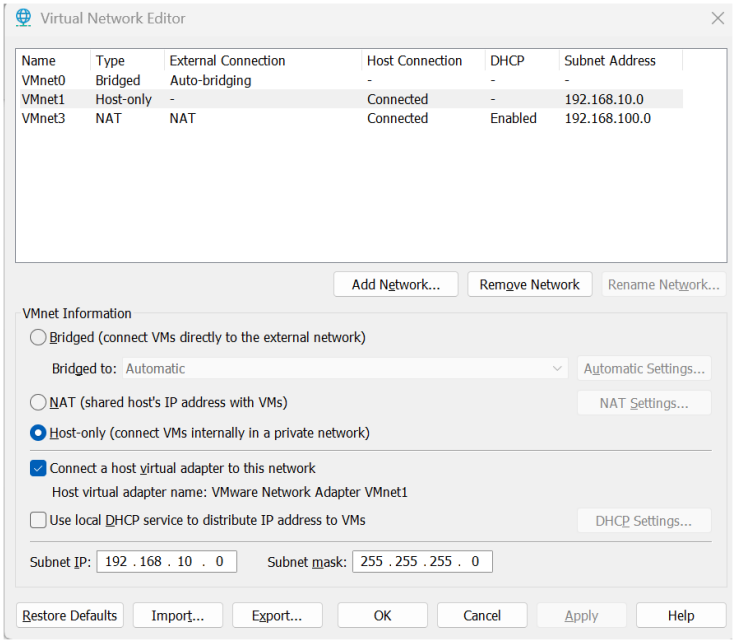
Network Configuration Table

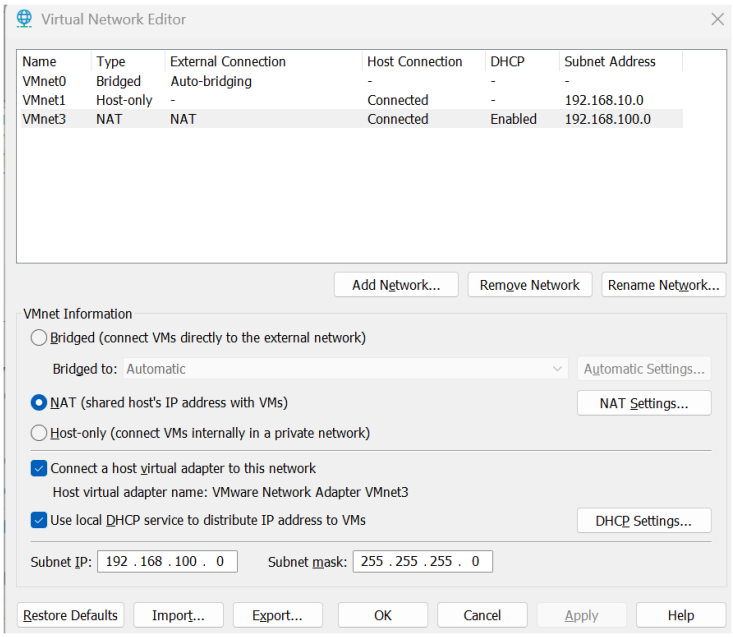
#	Machine Name	OS	VMnet1 (Host-only)	VMnet3 (NAT)	Primary Use
1	Ubuntu 64-bit	Ubuntu	192.168.10.20	192.168.100.135	Attacker (Lab 1)
2	Windows Server 2019	Windows Server	192.168.10.4	192.168.100.131	Web Server (Lab 1)
3	Windows 10	Windows 10	192.168.10.3	192.168.100.128	Admin (Lab 2)

1. Virtual Network Editor Setting

The VMware Virtual Network Editor is used to configure the virtual networks (VMnets) used by the lab VMs.

- VMnet1 — Host-only network, Subnet: 192.168.10.0, connected to host, no DHCP.
- VMnet3 — NAT network, Subnet: 192.168.100.0, DHCP enabled, connected to host.



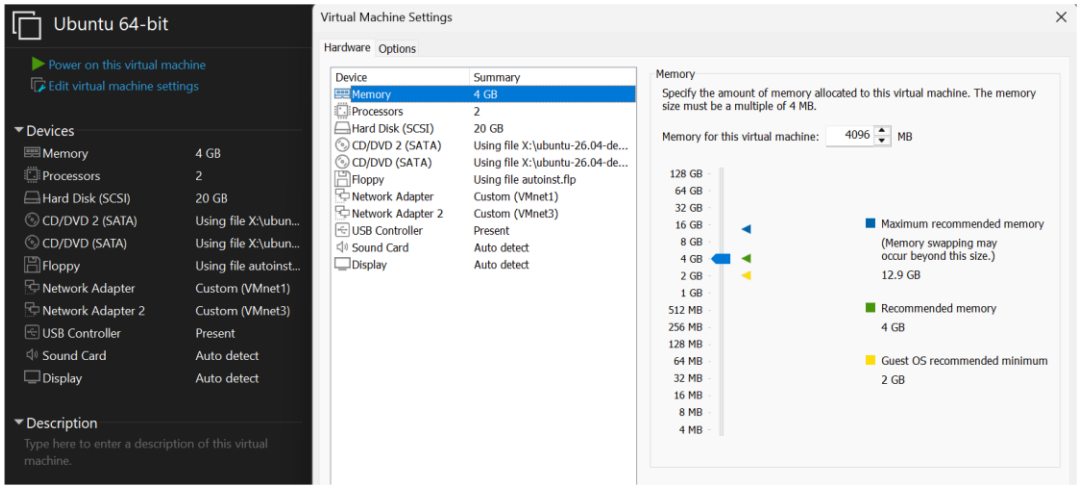


3. Network Setup (Windows/Windows Server/Ubuntu)

Open the Ubuntu VM used for the Operations Department.

It should have its network adapter set to:

- Custom (VMnet1) Host-only.
- Custom (VMnet3) NAT.



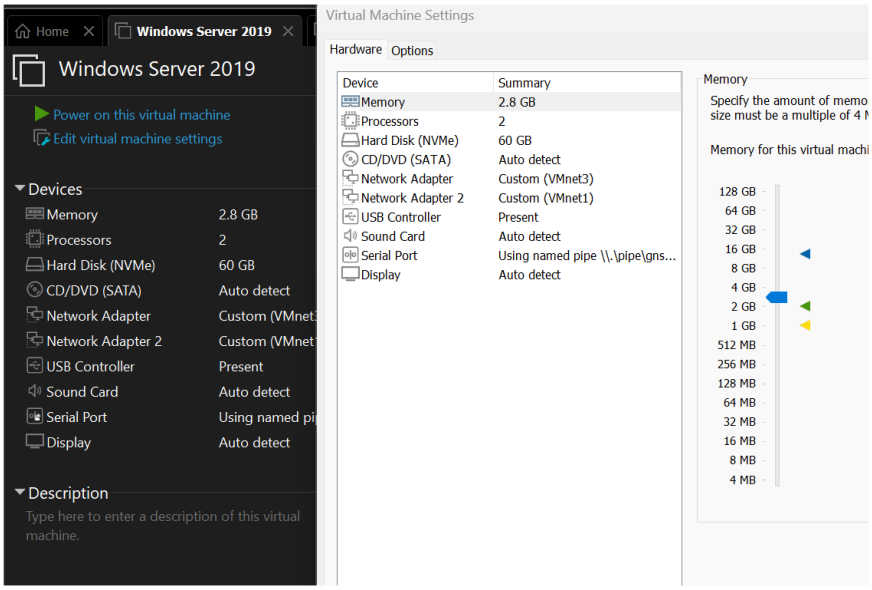
IP Check

```
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:8a:40:06 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.20/24 brd 192.168.10.255 scope global noprefixroute ens33
       valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe8a:4006/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 00:0c:29:8a:40:10 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c298a4010
    inet 192.168.100.135/24 brd 192.168.100.255 scope global dynamic noprefixrou
te ens37
```

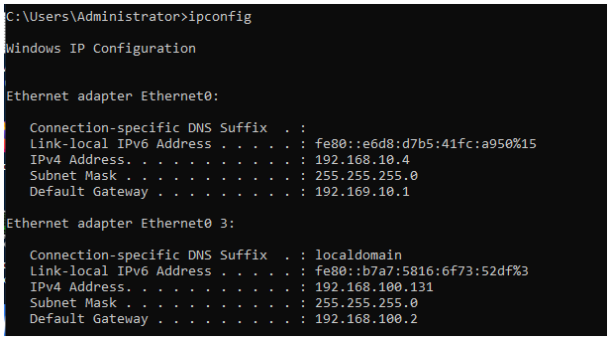
Open Windows server 2019 (Network Setup)

It should have its network adapter set to:

- Custom (VMnet1) Host-only.
- Custom (VMnet3) NAT.



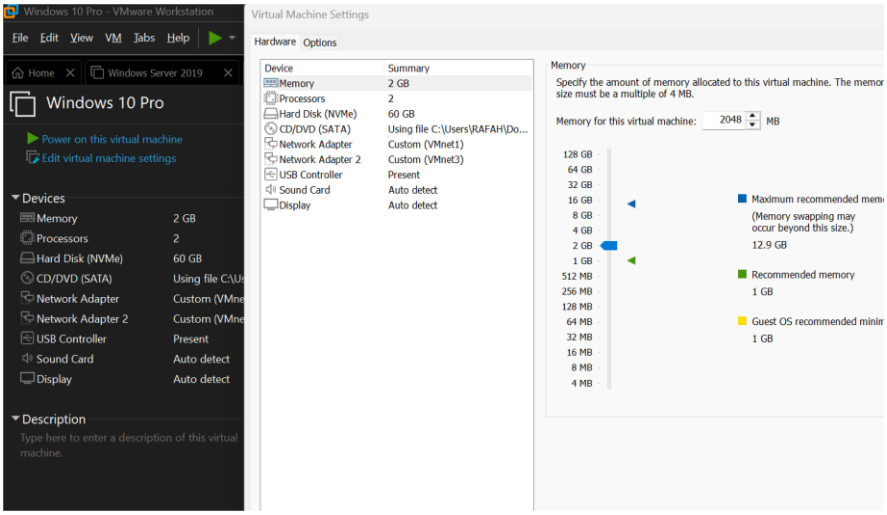
IP check



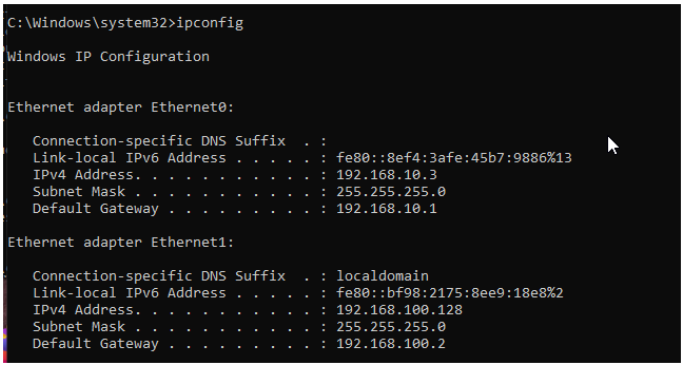
Open Windows 10 -> Admin Machine (Network Setup)

It should have its network adapter set to:

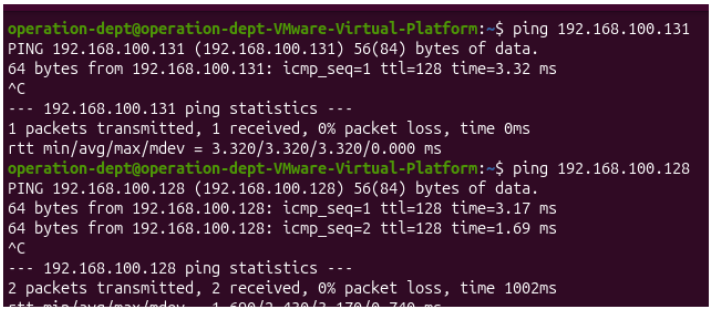
- Custom (VMnet1) Host-only.
- Custom (VMnet3) NAT.



IP check



Connectivity check Passed



LAB Setup Ready to Start…

Implementing Application Whitelisting Using AppLocker

Attackers often trick users into running malicious executables, scripts, or installers disguised as legitimate software. Application whitelisting is a defense strategy where only pre-approved (whitelisted) applications are allowed to run — everything else is blocked by default, even if it later gets onto the machine.

Objective:
Learn to implement application whitelisting using AppLocker, a Windows feature that lets an administrator control which apps, scripts, and installers users are allowed to run.

What you'll actually do in this lab:

1. Create AppLocker rules for different rule collections:
    - Executable rules (.exe, .com)
    - Windows Installer rules (.msi, .msp)
    - Script rules (.ps1, .bat, .cmd, .vbs, .js)
    - Packaged app rules (Windows Store apps)
2. Set rule enforcement mode (Enforce vs Audit only)
3. Start the Application Identity service (required for AppLocker to actually enforce anything)
4. Test the rules — try running a non-whitelisted app and confirm Windows blocks it
5. Confirm the block message the user sees

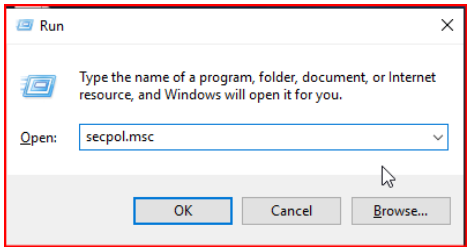
Machine: Windows Server 2019 (standalone)

Open Local Security Policy

Command:

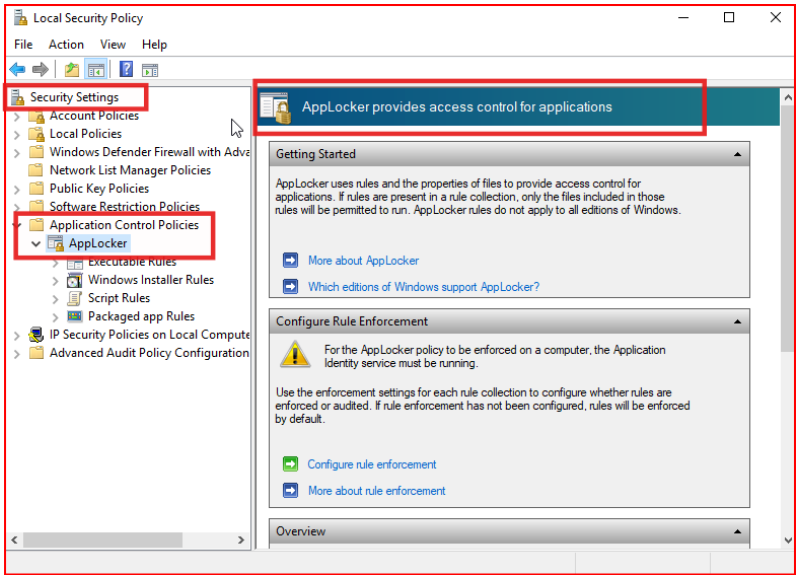
Win + R → secpol.msc → Enter

What you'll see: The Local Security Policy window opens.



Then navigate to:

Security Settings → Application Control Policies → AppLocker



Why this matters: This is where we'll create and manage all AppLocker rules (Executable, Script, Windows Installer, Packaged app) directly on this machine — no GPO/domain needed.

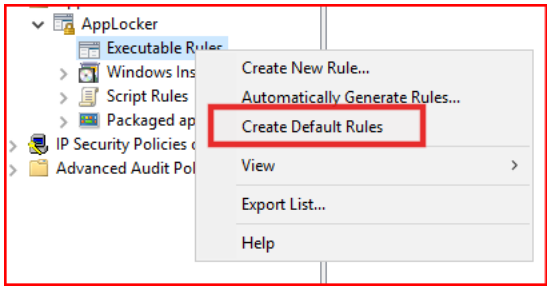
Expected output: You'll see the AppLocker node with an overview pane showing "Executable Rules," "Windows Installer Rules," "Script Rules," and "Packaged app Rules" — all showing "0" rules configured so far.

Create Default Executable Rules

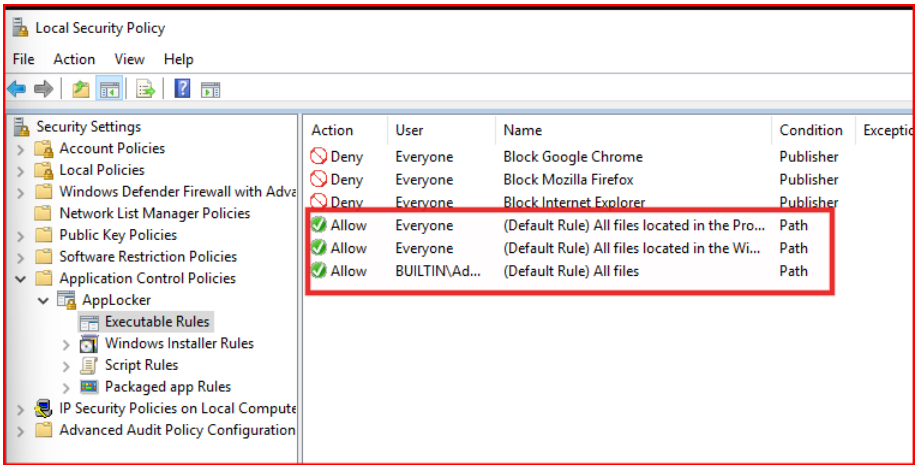
Why first: Before creating custom rules, we generate default rules so you don't lock yourself out of essential Windows/Program Files executables (like explorer.exe, services, etc.). Skipping this can freeze the system.

Command (mouse steps):

1. Right-click Executable Rules (left pane)
2. Select Create Default Rules



Expected output: 3 default rules appear in the right pane

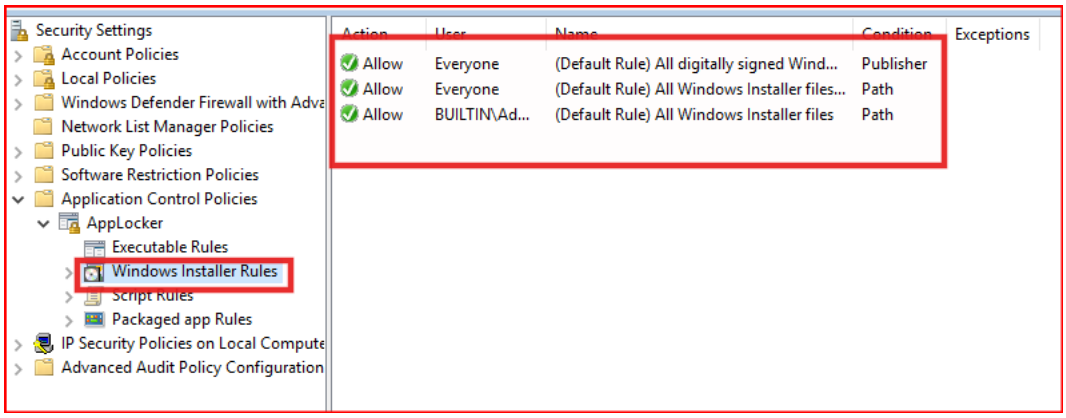


Create default rules for the other 3 rule collections

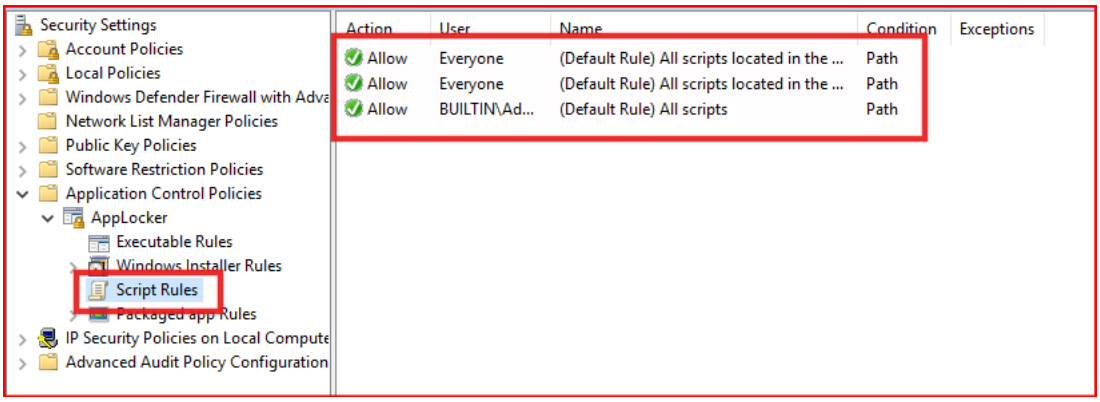
We only did Executable Rules. Now do the same for:

- Windows Installer Rules
- Script Rules
- Packaged app Rules

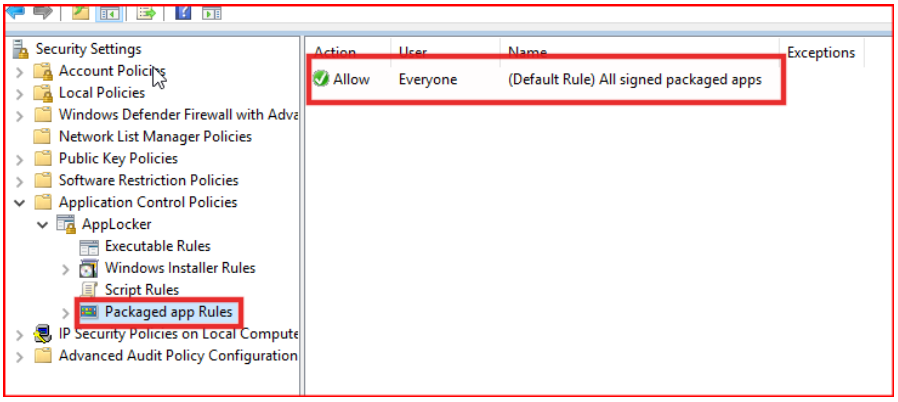
Right-click Windows Installer Rules → Create Default Rules



Right-click Script Rules → Create Default Rules



Right-click Packaged app Rules → Create Default Rules



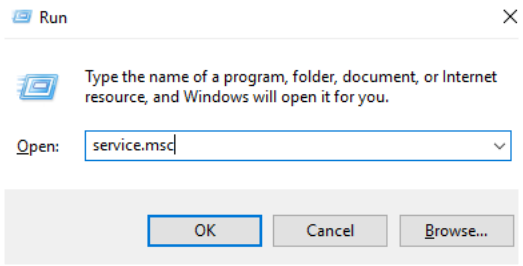
Why: Same reason as Executable Rules — without default rules in each collection, once enforcement is on, legitimate installers/scripts/store apps could get blocked too.

Start the Application Identity Service

Why: AppLocker rules exist but won't be enforced unless this service is running. This is the single most important step — skip it and nothing gets blocked, even with rules configured.

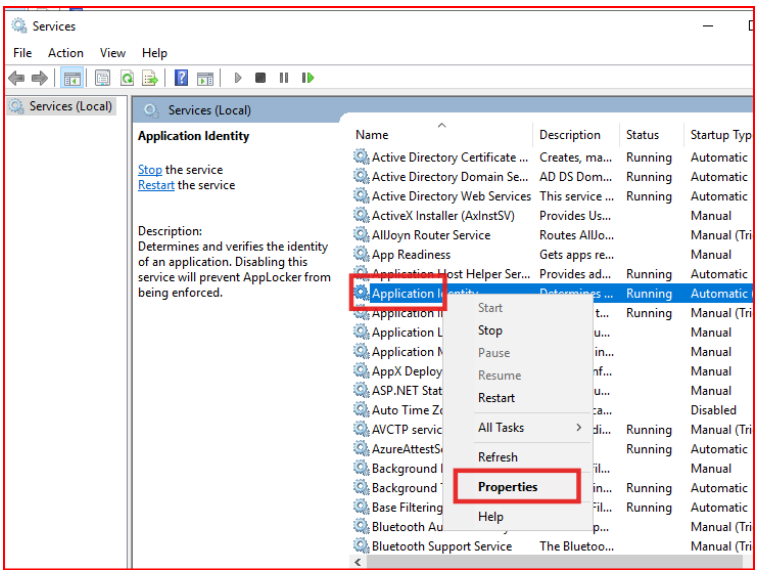
Command:

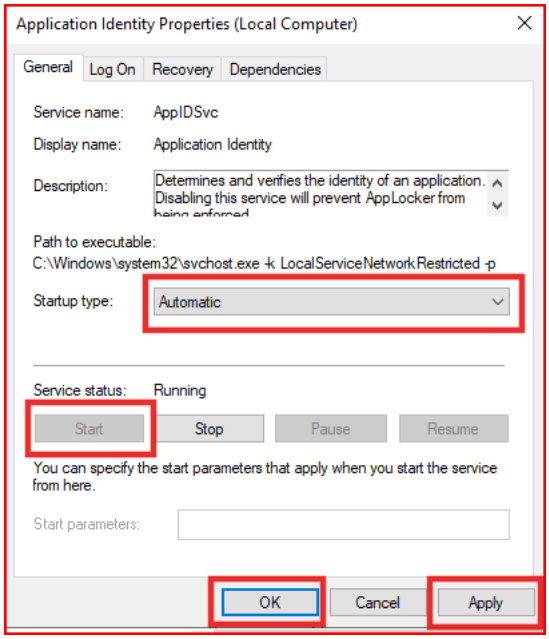
Win + R → services.msc → Enter



Then:

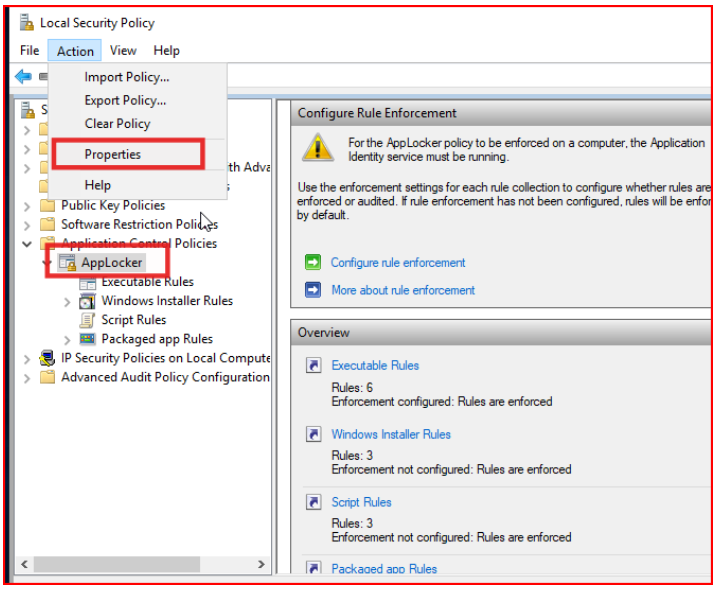
1. Find Application Identity in the list
2. Right-click → Properties
3. Set Startup type to Automatic
4. Click Start (if status isn't already "Running")
5. Click OK





Turn ON enforcement

Right now, your rules exist (Deny Chrome, Allow Program Files, etc.) but Windows is not actually applying them yet. This step switches them from "just sitting there" to "actively enforced."



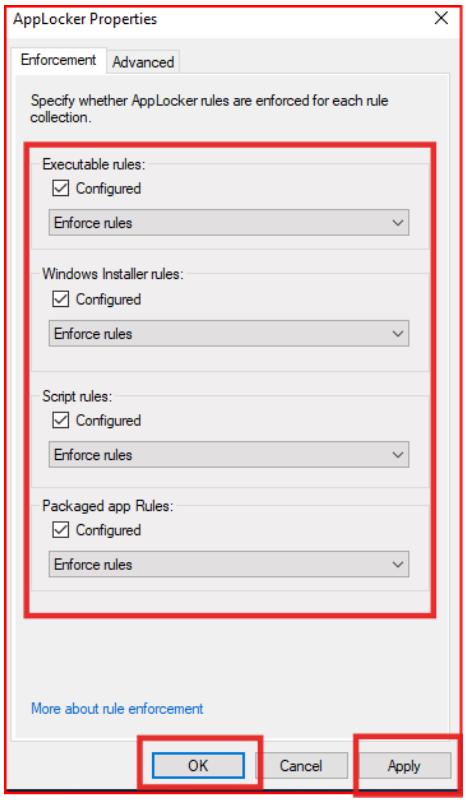
For each of these 4 tabs, one at a time:

1. Click the tab
2. Tick the checkbox that says Configured
3. A dropdown will appear/enable below it — make sure it says Enforce rules
4. Move to the next tab and repeat

After all, 4 tabs are done:

- Click Apply
- Click OK

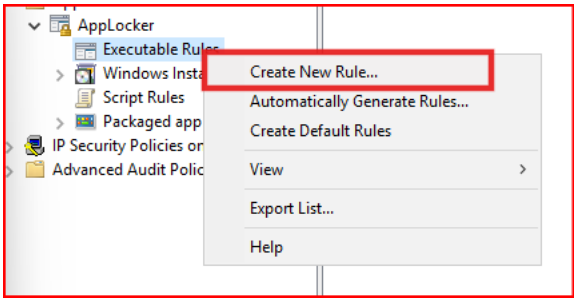
What does this look like when correct: every tab has the "Configured" checkbox ticked, and the dropdown next to it says, "Enforce rules" (not "Audit only").



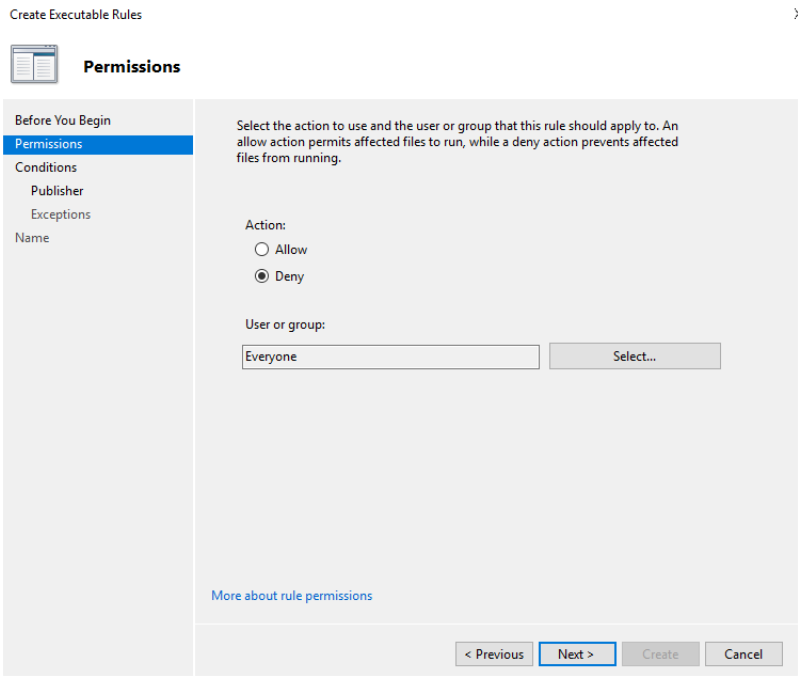
Create a test Deny rule + verify block

Step 6a: Create a Deny rule for Calculator

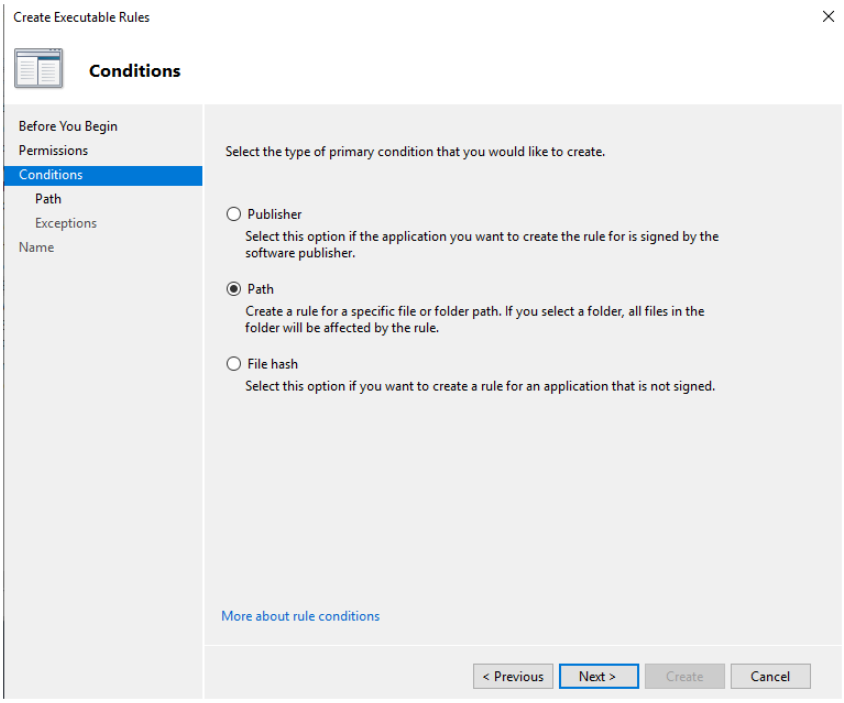
1. In Local Security Policy, right-click Executable Rules → Create New Rule...



1. Click Next on the welcome screen
2. Select Deny, User/Group: Everyone → Next

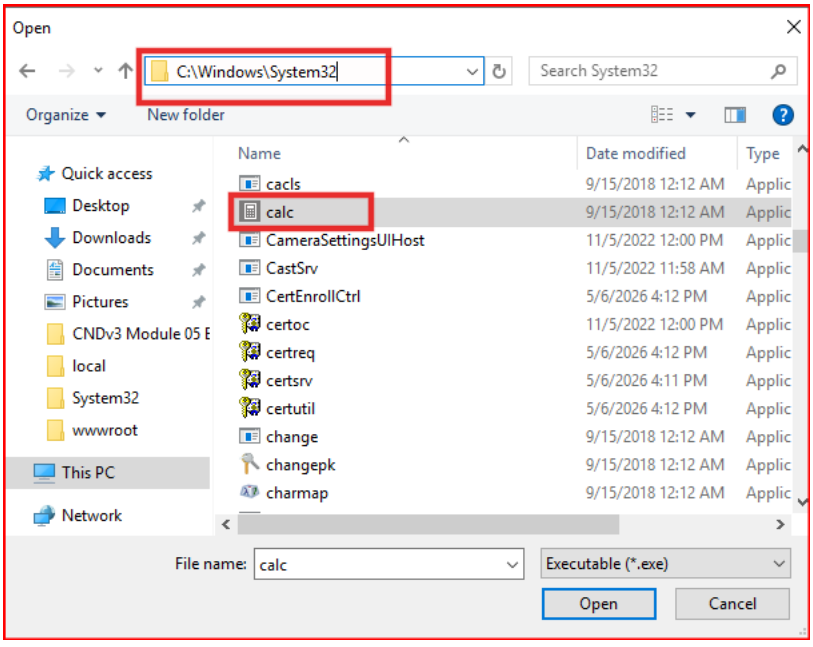


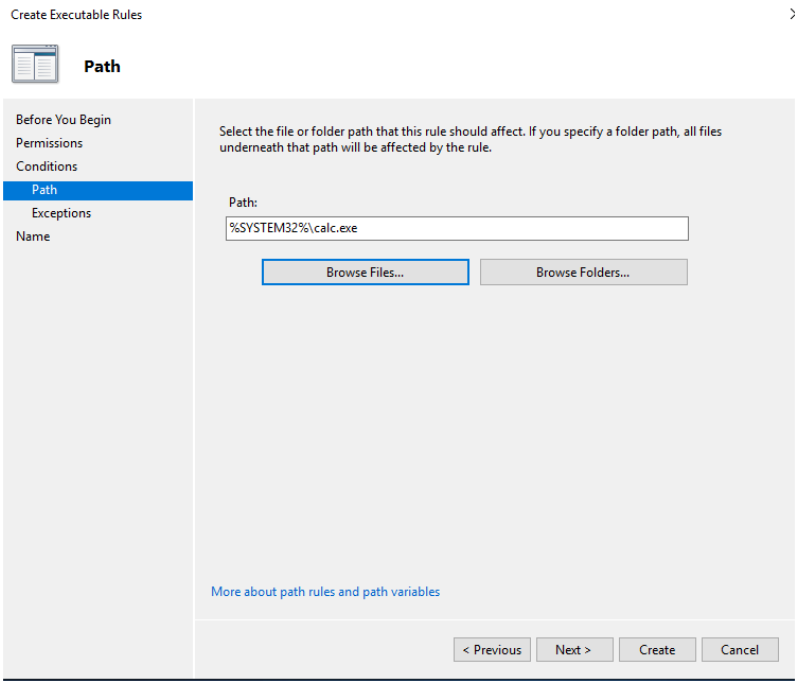
1. Choose condition type: Path (simplest option) → Next



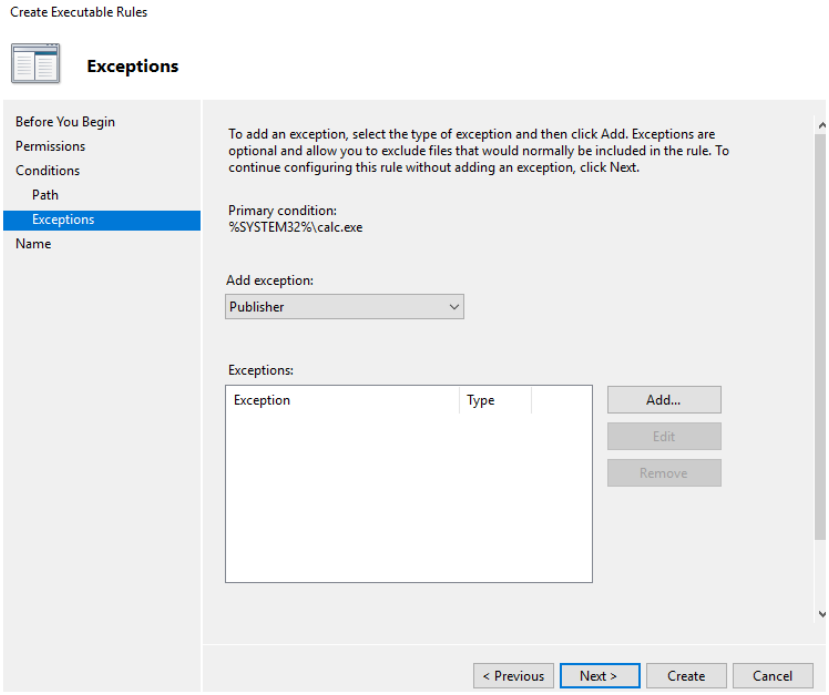
1. Click Browse Files... and navigate to:

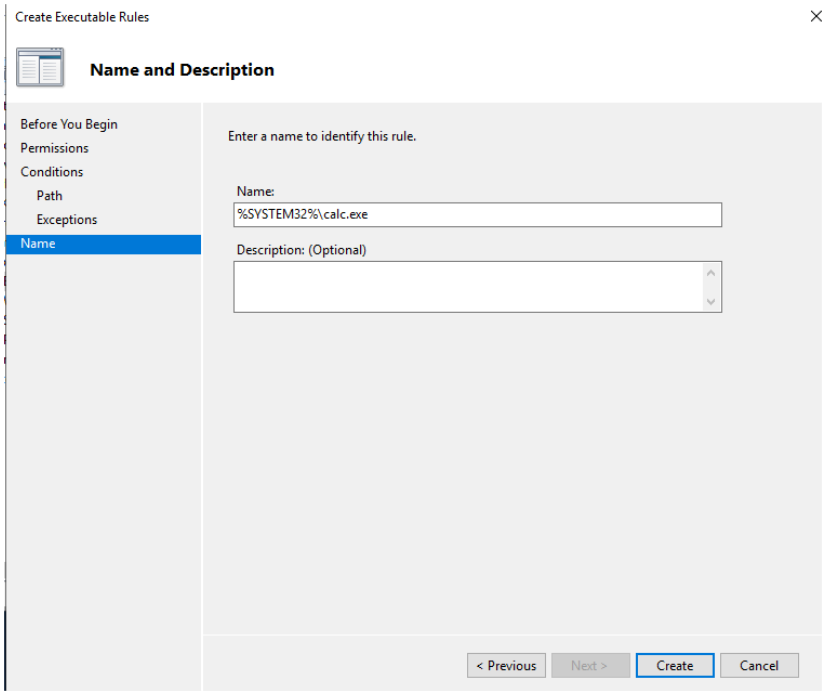
C:\Windows\System32\calc.exe



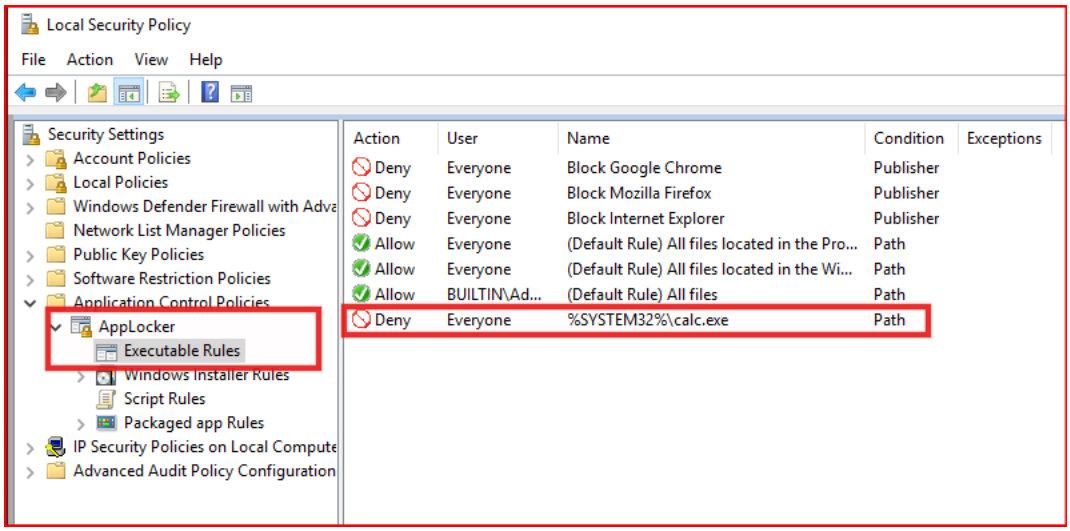


6. Click Next → Next (skip exceptions) → Create





Expected output: A new Deny rule appears in Executable Rules for Calculator (or whichever app you picked).



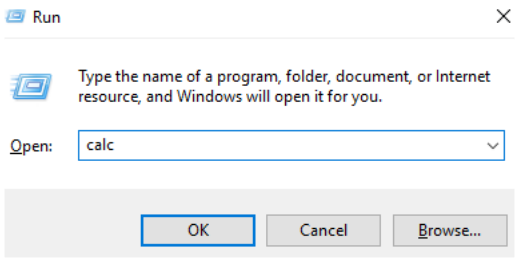
Try to open Calculator and confirm it's blocked

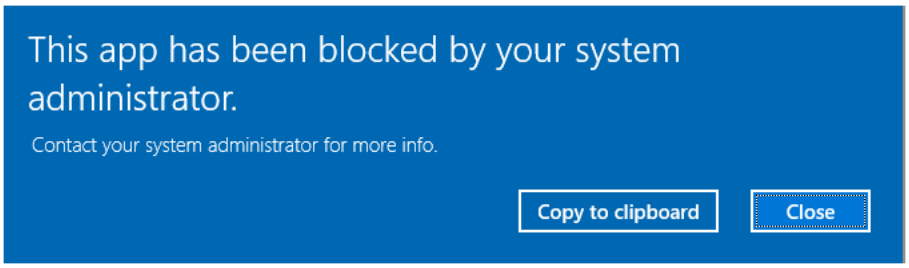
Command:

1. Press Win + R
2. Type calc → Enter

Expected output: Calculator should fail to open, and you should see a message like:

"This app has been blocked by your system administrator."





Implementing Application Whitelisting Using Software Restriction Policies (SRP)

SRP is an older Windows whitelisting technology (predates AppLocker) but still useful — it works on all Windows editions, including Home, so it's actually more flexible than AppLocker in terms of compatibility.

Objective:
Learn to control which applications are allowed to run using Software Restriction Policies, an alternative whitelisting mechanism to AppLocker.

What you'll do:

1. Open Local Security Policy → Software Restriction Policies
2. Create a new SRP node (if none exists)
3. Set the default security level (Unrestricted vs Disallowed)
4. Create additional rules (Path Rule, Hash Rule, or Certificate Rule) to allow/deny specific apps
5. Test by trying to run a blocked app and confirming it's denied

Key difference from AppLocker:

- SRP applies to all users by default (less granular than AppLocker's per-user/group targeting)
- SRP has 4 rule types: Hash, Certificate, Path, Network Zone (AppLocker has similar but adds Publisher-based rules)
- SRP works on any Windows edition — this is why it's a good fit for your Windows 10 Home machine too, if you want to try it there later

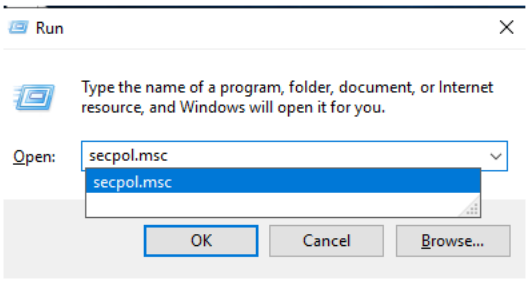
Machine to use: Same Windows Server 2019 (standalone) keeps things consistent, though this one will also work fine on your Windows 10 Home machine if you'd rather test it there.

Windows Server 2019 (standalone)

Open Software Restriction Policies node

Command:

Win + R → secpol.msc → Enter

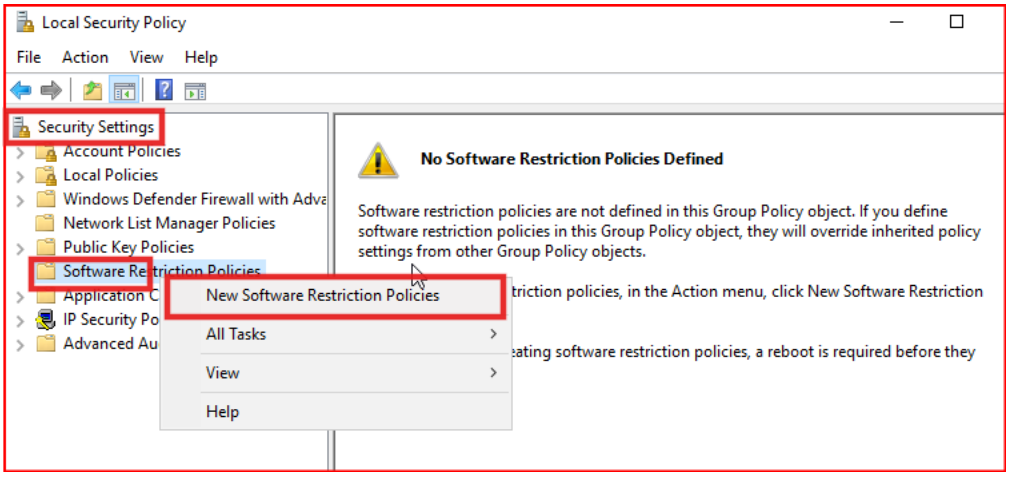


Then navigate to:

Security Settings → Software Restriction Policies

What you'll see: Likely empty/no policies defined yet — you'll see a message like "No Software Restriction Policies Defined."

Expected action: Right-click Software Restriction Policies → New Software Restriction Policies



Check the Default Security Level

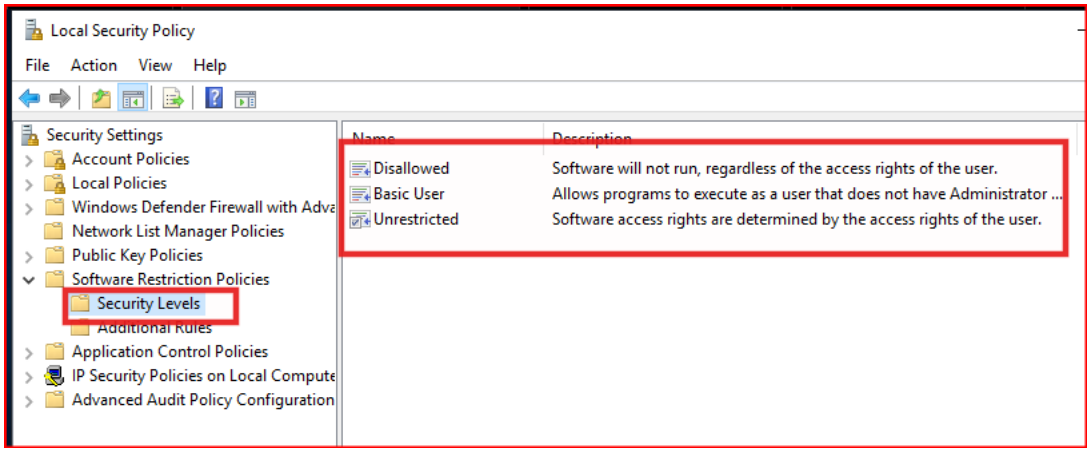
Why: This decides whether apps are allowed by default (Unrestricted) or blocked by default (Disallowed). We need to know the starting point before adding rules.

Command:

1. Click Security Levels (left pane)
2. You'll see two options in the right pane:
    - Disallowed
    - Unrestricted (this one usually has a small checkmark/black circle showing it's the current default)

Expected output: "Unrestricted" is set as default — meaning all apps run unless we specifically block them (this matches how the lab wants us to test — same approach as Lab 1's Deny rules).

Leave it on Unrestricted — don't change it yet. We'll add a specific Path Rule to block one app next, same pattern as Lab 1.

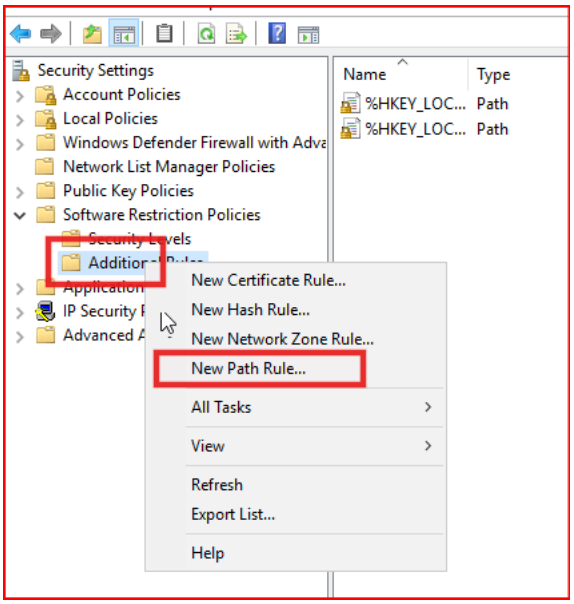


Create a Path Rule to block a specific app

Why: Since default is "Unrestricted," we now add a specific rule to block one app — this proves SRP can restrict execution even when the general policy allows everything else.

Command:

1. Right-click Additional Rules (left pane)
2. Select New Path Rule...

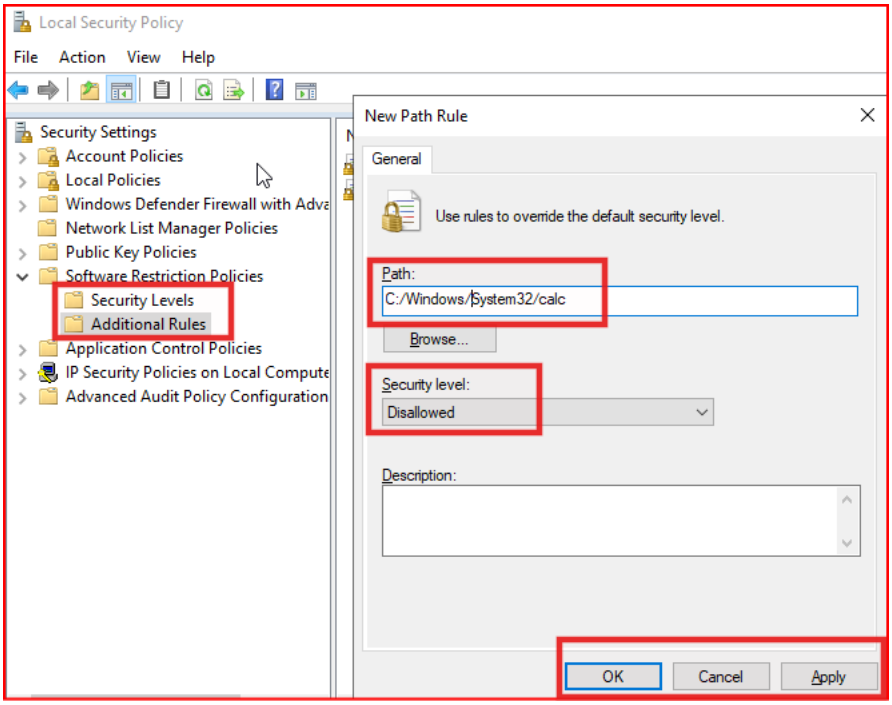


3. In the Path field, click Browse... and navigate to:

   C:\Windows\System32\win32calc

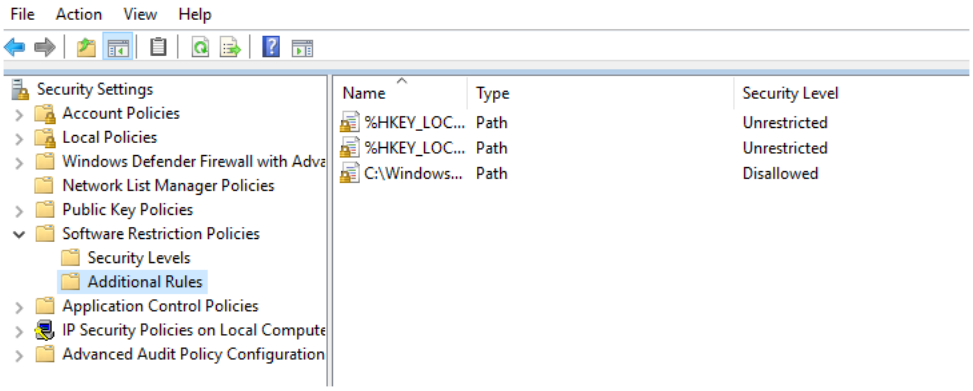
   (same app as Lab 1, for consistency — or pick Notepad if you prefer a different one)

4. Set Security level dropdown to Disallowed
5. Add a description (optional): "Block Calculator via SRP"
6. Click OK



Expected output: A new rule appears under Additional Rules showing:

- Path: C:\Windows\System32\calc.exe
- Security Level: Disallowed

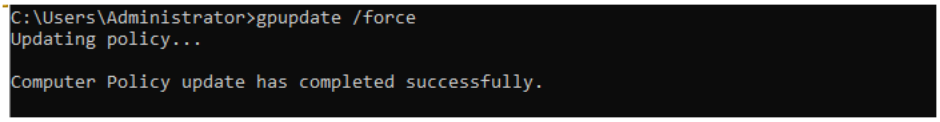


Force policy refresh

Why: SRP changes sometimes need a manual refresh to take effect immediately (unlike AppLocker which we tested directly).

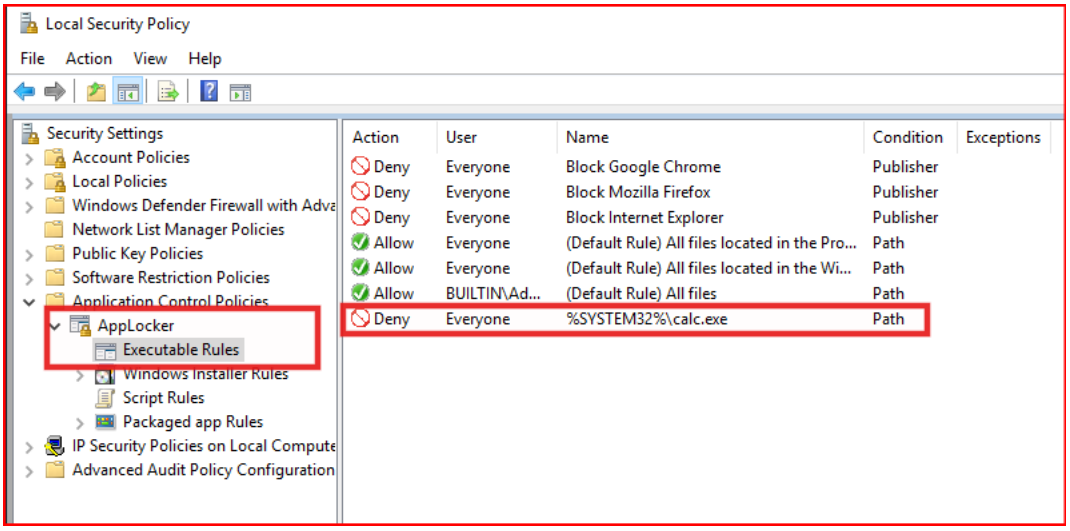
Command:
Open Command Prompt as Administrator, then run:

gpupdate /force

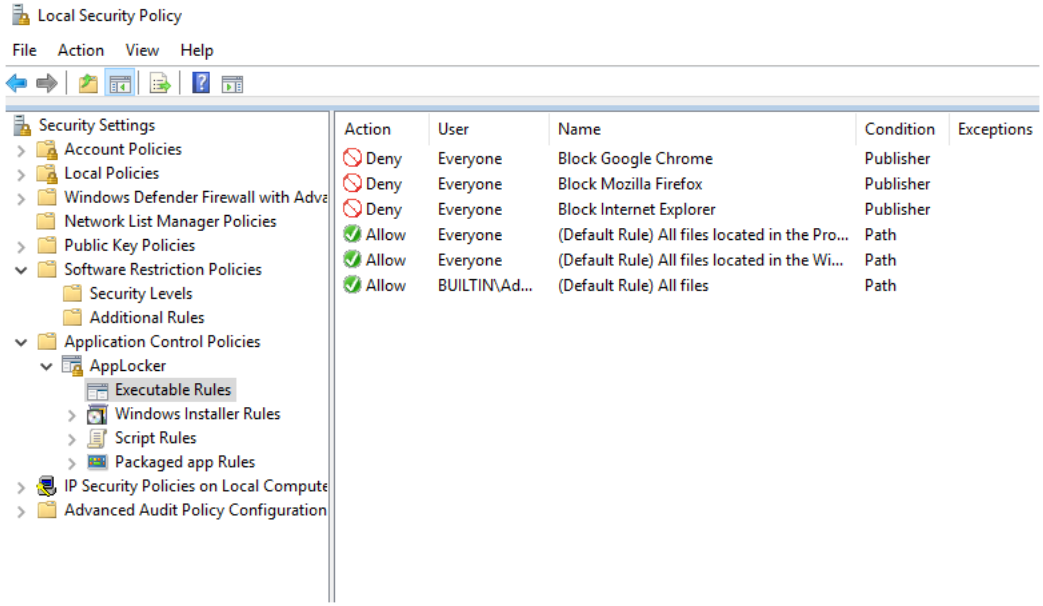


Now before checking the calculator navigate to place where we made the executable rule for calc and delete that rule

Before



After



Now restart the machine first…..

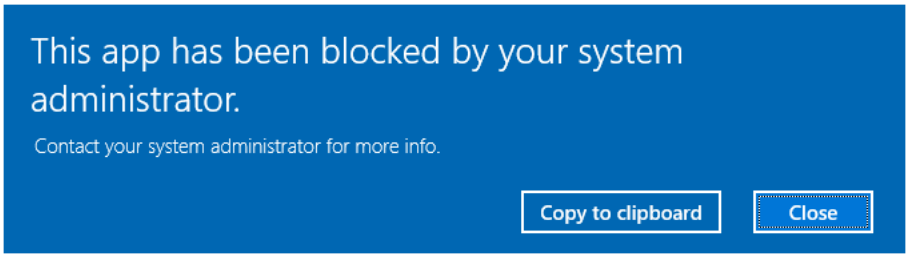
Test the block — try running Calculator

Command:

1. Press Win + R
2. Type calc → Enter

Expected output: Calculator should be blocked, likely with a message like:

"This program is blocked by group policy. For more information, contact your system administrator."



Implementing Application Security Using Firejail Sandbox

Untrusted or unreliable applications (downloaded tools, unknown scripts, beta software) can pose risk if run directly on a system — they might crash the system, leak data, or behave maliciously. Firejail is a Linux sandboxing tool that isolates an application's access to the file system, network, and devices, so even if it misbehaves, the damage stays contained.

Objective:
Learn to install and use Firejail to run applications inside an isolated sandbox, reducing risk from untrusted programs.

What you'll do:

1. Install Firejail on your Linux VM (likely your Ubuntu "Attacker" machine from earlier setup)
2. Verify installation
3. Run a normal application (e.g., a browser or file manager) inside Firejail's sandbox
4. Confirm the app is sandboxed (check restricted filesystem/network view)
5. Test that sandboxed app changes don't persist outside the sandbox
6. (Optional/advanced) Create a custom Firejail profile for a specific app

Machine to use: Your Ubuntu VM (the Linux machine you set up earlier — 192.168.10.20)

Update system and install Firejail

Command (on Ubuntu terminal):

sudo apt update

sudo apt install firejail -y

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ sudo apt update -y
[sudo: authenticate] Password:
Hit:1 http://pk.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://pk.archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:3 http://pk.archive.ubuntu.com/ubuntu resolute-backports InRelease
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ sudo apt install firejail -y
Installing:
  firejail

Installing dependencies:
  firejail-profiles  libxcb-icccm4  libxcb-image0  libxcb-keysyms1  libxcb-render-util0  libxcb-xkb1  libxcb-xv0  xserver-xephyr

Summary:
  Upgrading: 0, Installing: 9, Removing: 0, Not Upgrading: 97
  Download size: 1,668 kB
  Space needed: 6,428 kB / 9,254 MB available

Get:1 http://pk.archive.ubuntu.com/ubuntu resolute/universe amd64 firejail amd64 0.9.72-2ubuntu5 [414 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu resolute/universe amd64 firejail-profiles all 0.9.72-2ubuntu5 [113 kB]
```

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ firejail --version
firejail version 0.9.72

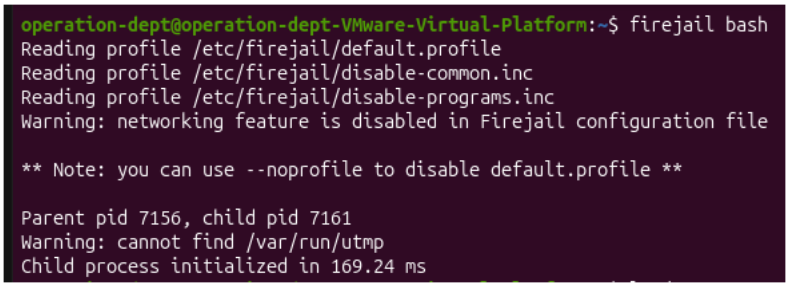
Compile time support:
        - always force nonewprivs support is disabled
        - AppArmor support is enabled
        - AppImage support is enabled
        - chroot support is enabled
        - D-BUS proxy support is enabled
        - file transfer support is enabled
        - firetunnel support is disabled
        - IDS support is enabled
        - networking support is enabled
        - output logging is enabled
        - overlayfs support is disabled
        - private-home support is enabled
```

Run a simple test app inside Firejail

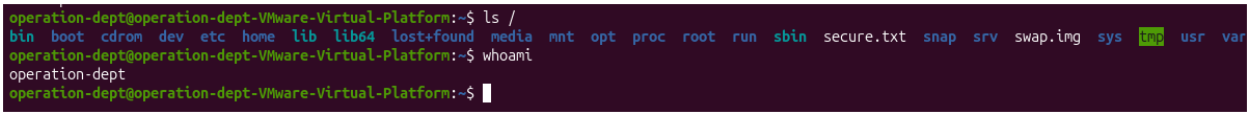
Why: Confirms Firejail can actually launch and sandbox an application before we test anything more complex.

Use bash — guaranteed to exist

Firejail sandbox launched successfully — profile loaded, child process initialized, and you're running as operation-dept (not root) inside the sandbox.



ls / -> shows a restricted/whitelisted view of the filesystem (fewer folders than a normal ls / would show outside the sandbox).



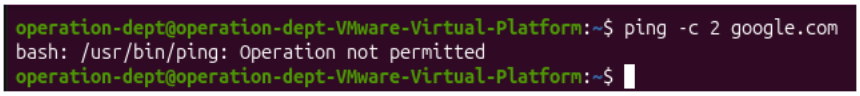
Prove the sandbox actually isolates something (not just launches)

Right now, we've confirmed Firejail *runs* an app — but we haven't proven it *restricts* anything yet. Let's verify real isolation using two checks: networking and filesystem writes.

Check A: Networking is disabled inside the sandbox

While still inside your firejail bash session, run:

ping -c 2 google.com



Expected output: This should fail (no network), since your earlier screenshot already showed:

"Warning: networking feature is disabled in Firejail configuration file"

Check B: Filesystem write isolation

Still inside the sandbox, run:

echo "test from sandbox" > /tmp/sandboxtest.txt

cat /tmp/sandboxtest.txt

Then exit the sandbox:

exit

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ echo "testing from sandbox" > /tmp/sandboxtest.txt
operation-dept@operation-dept-VMware-Virtual-Platform:~$ cat /tmp/sandboxtest.txt
testing from sandbox
operation-dept@operation-dept-VMware-Virtual-Platform:~$ exit
exit

Parent is shutting down, bye...
```

Now, outside the sandbox (normal terminal), check if that file exists:

cat /tmp/sandboxtest.txt

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ cat /tmp/sandboxtest.txt
testing from sandbox
operation-dept@operation-dept-VMware-Virtual-Platform:~$
```

Why this matters: If Firejail is using private-tmp or similar isolation, the file created inside the sandbox won't exist outside it — proving true isolation. If it does persist, that tells us this particular profile shares /tmp (which is also useful info to document).

Prove stronger isolation using --private flags

Let's now test genuinely stricter sandboxing, so your lab shows the *tunable* nature of Firejail (not just default behavior).

Command:

firejail --private bash

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ firejail --private bash
Reading profile /etc/firejail/default.profile
Reading profile /etc/firejail/disable-common.inc
Reading profile /etc/firejail/disable-programs.inc
Warning: networking feature is disabled in Firejail configuration file

** Note: you can use --noprofile to disable default.profile **

Parent pid 7314, child pid 7316
Warning: cannot find /var/run/utmp
Child process initialized in 180.33 ms
```

Inside this sandbox, run:

ls ~

whoami

echo "private test" > ~/privatetest.txt

cat ~/privatetest.txt

Then exit:

Exit

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ ls ~
operation-dept@operation-dept-VMware-Virtual-Platform:~$ whoami
operation-dept
operation-dept@operation-dept-VMware-Virtual-Platform:~$ echo "private test" > ~/privatetest.txt
operation-dept@operation-dept-VMware-Virtual-Platform:~$ cat  ~/privatetest.txt
private test
operation-dept@operation-dept-VMware-Virtual-Platform:~$ exit
exit

Parent is shutting down, bye...
```

Outside the sandbox, check:

cat ~/privatetest.txt

```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ cat ~/privatetest.txt
cat: /home/operation-dept/privatetest.txt: No such file or directory
operation-dept@operation-dept-VMware-Virtual-Platform:~$
```

Expected: Inside the sandbox, ls ~ should show an empty/different home directory (not your real files). And after exiting, ~/privatetest.txt should not exist on the real system — proving --private mode truly isolates the home directory.

List active sandboxes (monitoring aspect)

Why: In real use, an admin needs to see what sandboxes are currently running — this is part of managing Firejail, not just launching it.

Command (run in a normal terminal, outside any sandbox):

firejail –list

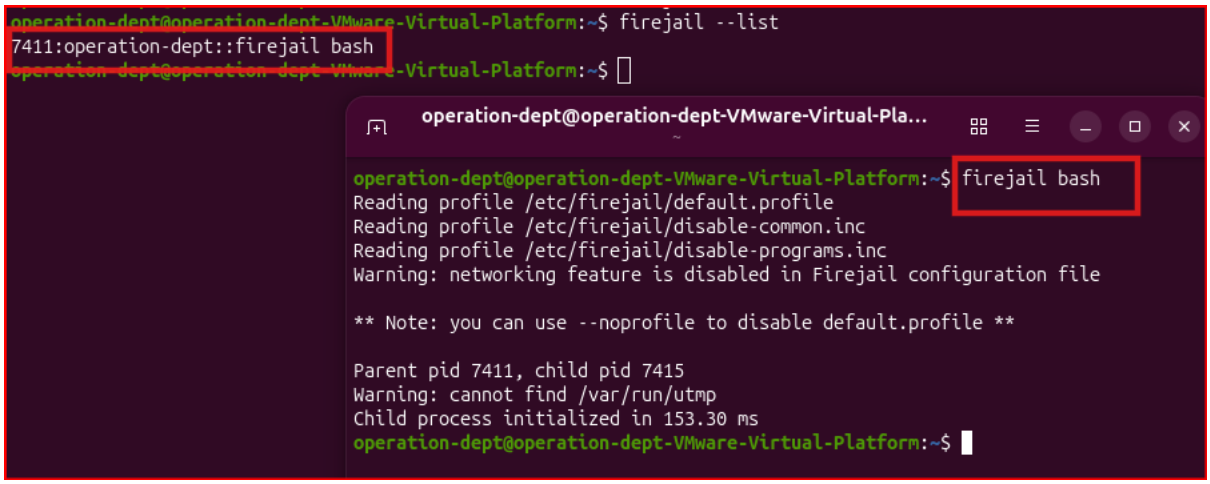
```
operation-dept@operation-dept-VMware-Virtual-Platform:~$ firejail --list
```

Expected output: If nothing is currently sandboxed, it'll show empty.

So, let's test it properly:

Do this in sequence:

1. Open a second terminal window (or new tab)
2. In terminal 1: run firejail bash (keep it open/running)
3. In terminal 2: run firejail --list



Key learning outcomes:

- Firejail sandboxes apps by restricting network, process visibility, and filesystem access based on per-app profiles (/etc/firejail/*.profile)
- Default profiles share /tmp between host and sandbox unless --private or --private-tmp is used
- --private creates a fully isolated, temporary home directory — nothing persists after the sandbox closes
- Admins can monitor active sandboxes system-wide using firejail --list
- Real-world testing sometimes requires troubleshooting (missing apps, typos, root vs user context) — you correctly diagnosed and resolved each one

Implementing Application Security using URLScan Filter

Objective: Configure URLScan filter on IIS to block XSS attacks

Why?

- Windows Server holds IIS (Internet Information Services)
- This is the web server we'll configure
- LuxuryTreats website will run here

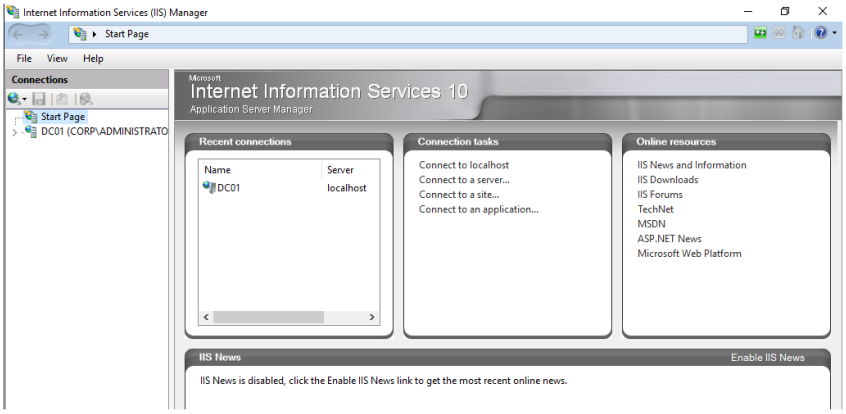
Open IIS Manager

1. Press Windows Key or click Start menu

2. Type "IIS Manager" in search box

3. Click "Internet Information Services (IIS) Manager"

4. Wait for IIS Manager window to open



Expand Server in Connections Pane

1. Look at LEFT panel "Connections"

2. Click the ARROW/PLUS (+) next to server name (DC01)

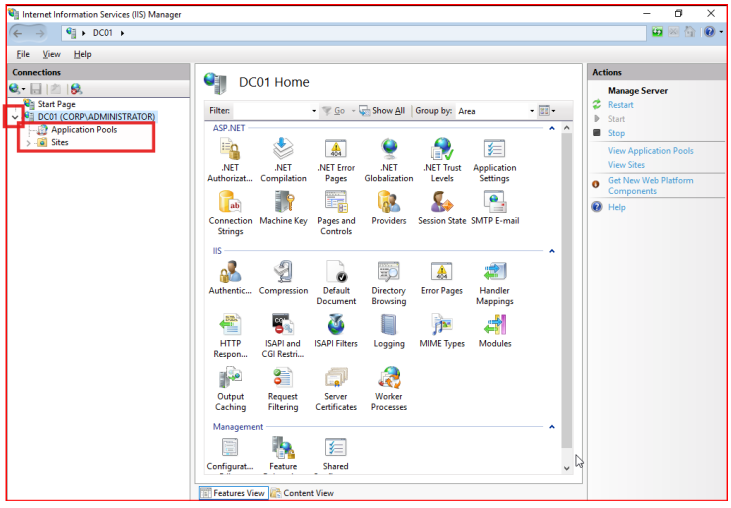
3. This will expand the server tree

4. You should see:

- Application Pools

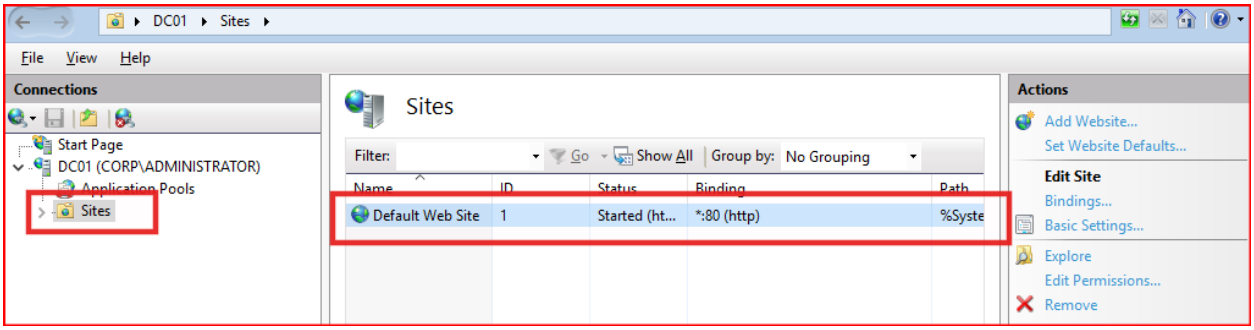
- Sites

- Other options



Click on "Sites" in Left Panel

You'll see "Default Web Site"



Select Default Web Site

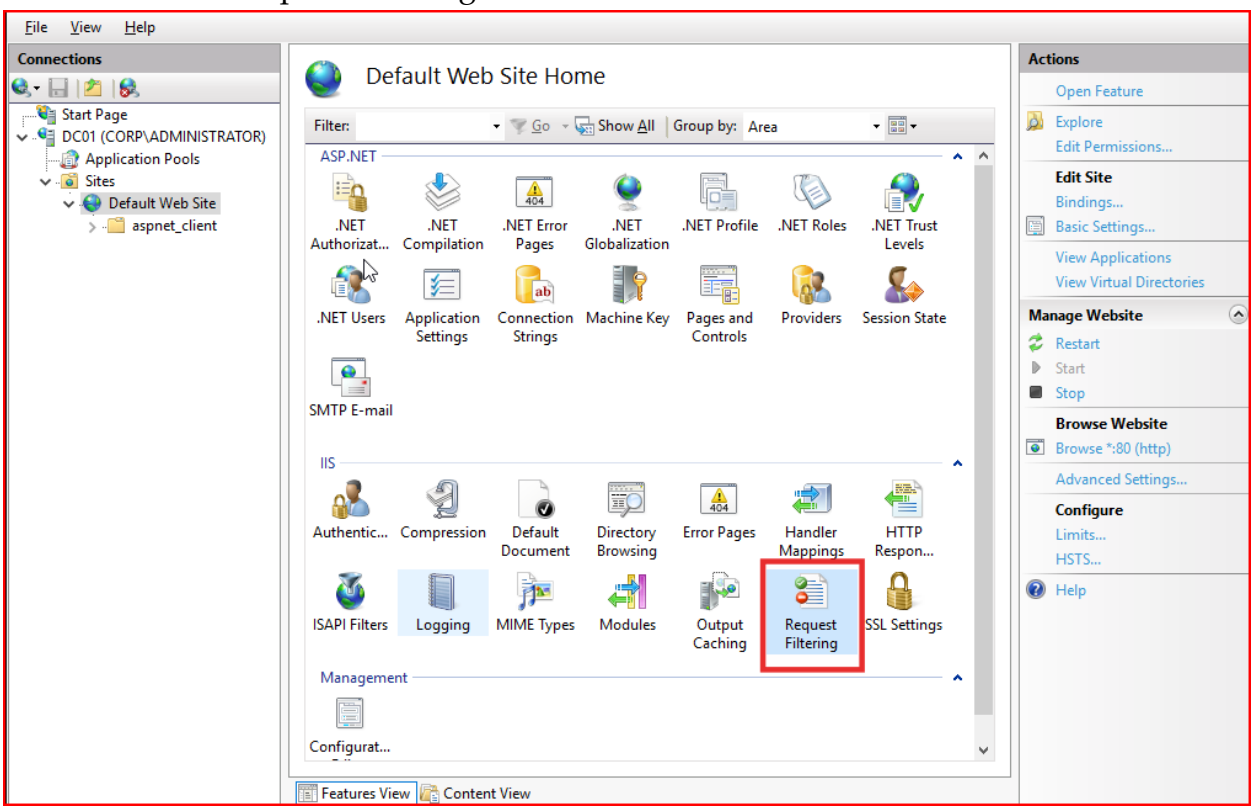
1. In the left panel, click "Default Web Site"

2. The middle panel will show all configuration options

3. Select "Default Web Site" in left panel

4. In middle panel, find "Request Filtering"

5. Double-click "Request Filtering"



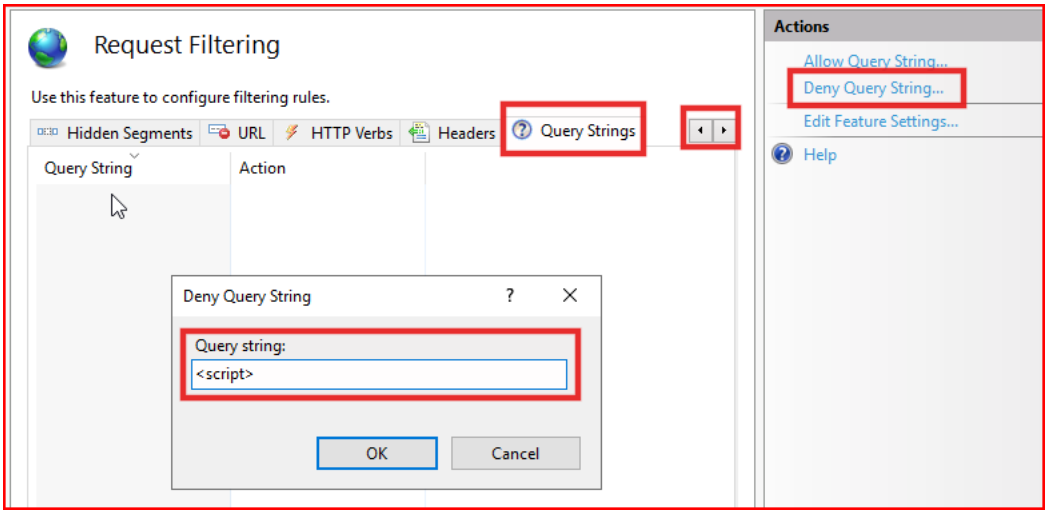
Configure Request Filtering Rules

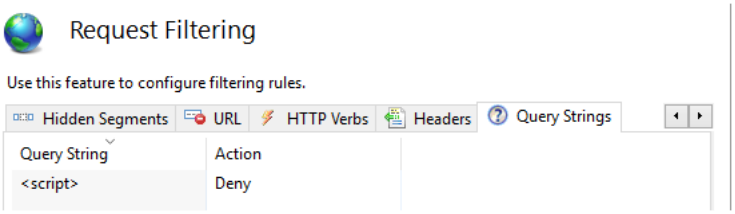
1. Look for "Denied Query Strings" tab/option

2. OR look for "Request Filtering Rules" section

3. We'll add a rule to block XSS patterns

4. XSS pattern example: <script> tags





Apply Changes - Restart IIS

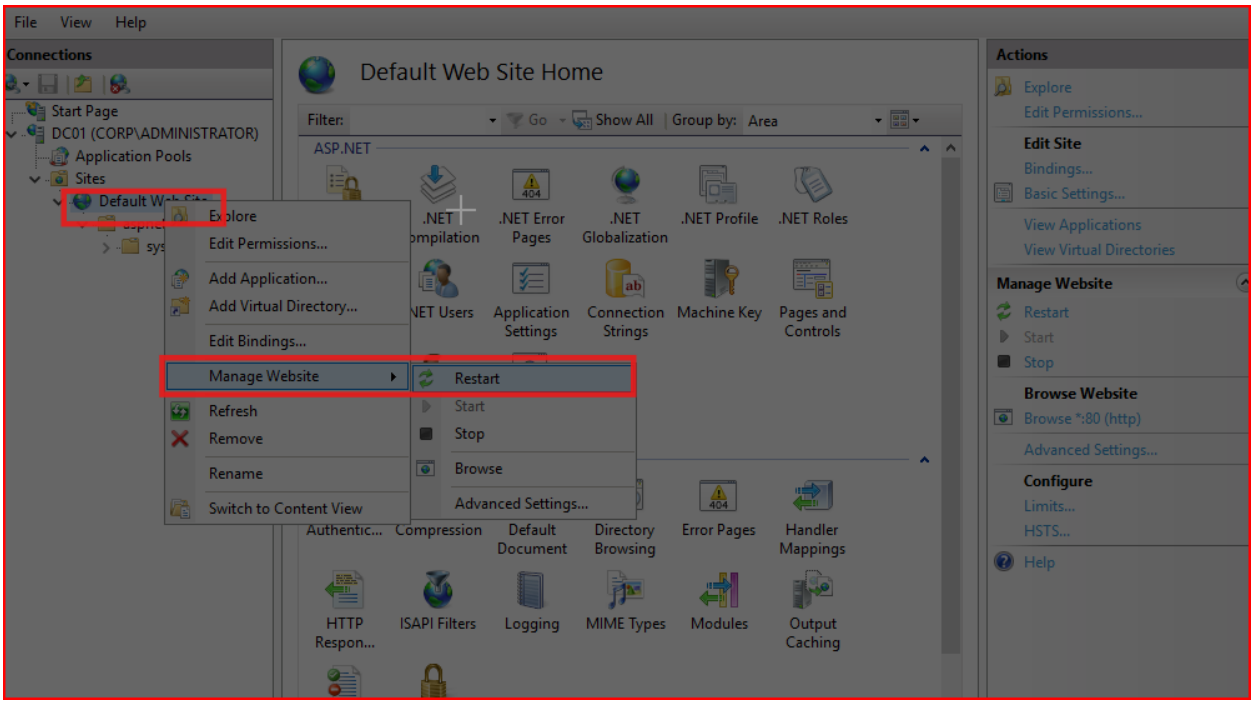
1. Close Request Filtering window

2. Go back to IIS Manager main window

3. Right-click on "WEBSERVER" in left panel

4. Select "Restart"

5. OR click "Restart" button in right Actions panel



Test XSS Attack - BEFORE Filter

Machine: UBUNTU (Attacker Machine)

Task:

1. Switch to Ubuntu VM (192.168.100.135)

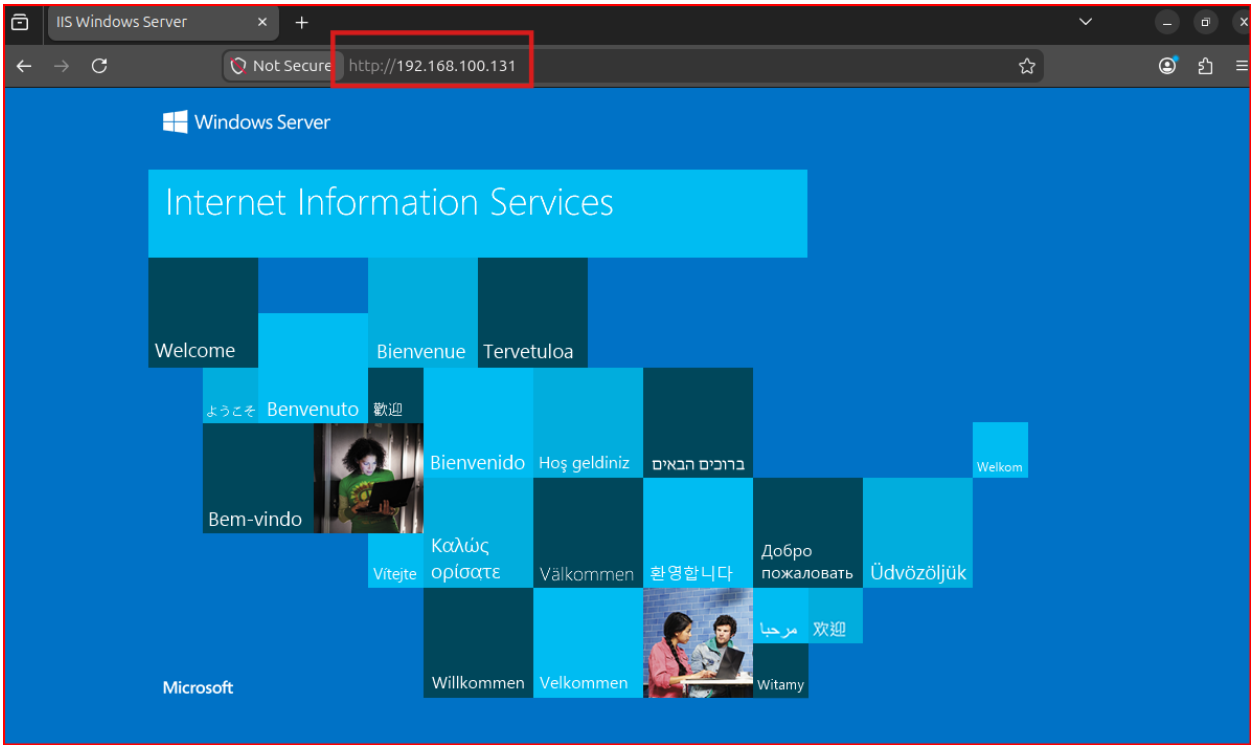
2. Open Firefox or Chrome browser

3. Type in address bar:

   http://192.168.100.131

4. Press Enter

5. Default Web Site should load



Why?

- We need to test if website is accessible first
- Then we'll try XSS attack with <script> tag
- Compare results: before and after filtering

Test XSS Attack with <script> Tag

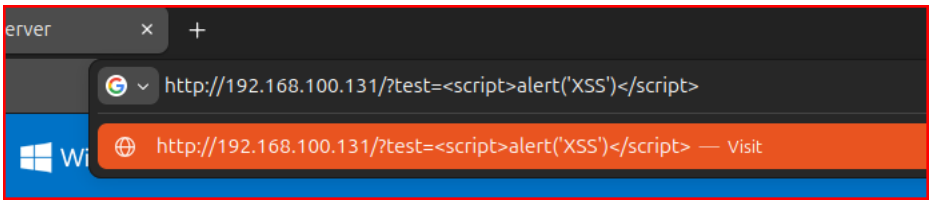
1. In address bar, modify URL to include <script> tag

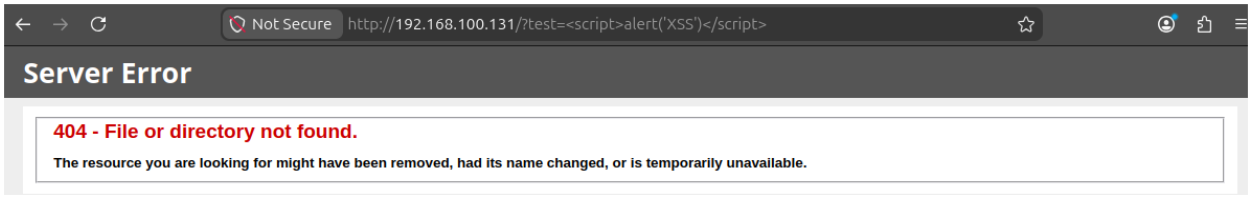
2. Change URL to:

   http://192.168.100.131/?test=<script>alert('XSS')</script>

3. Press Enter

4. Watch for response





Request Filtering Rule: SUCCESSFULLY APPLIED

- BEFORE FILTER (Without blocking):
- <script> tag would execute
- XSS attack succeeds
- Browser alert would show

AFTER FILTER (With blocking):

- URL contains <script>
- Request Filtering detects pattern
- Server returns HTTP 404
- Attack BLOCKED
- No script execution

Application Security using Windows Sandbox

"Implementing Application Security using Sandbox Technology"

The goal of this lab is to learn how to safely examine and test suspicious or untrusted files/applications (like malware, unknown executables, or files from unverified sources) without risking the real operating system.

In real-world cybersecurity, before running any unknown file on a production system, security professionals first test it in an isolated environment to observe what it does — without letting it damage, infect, or spread on the actual machine.

WHY do we need this lab?

1. **Real-world scenario:** Employees often download files from email attachments, USBs, or the internet. If one of those files is malware, opening it directly on a real system could:
    - Steal data
    - Encrypt files (ransomware)
    - Spread across the network
    - Damage the OS
2. **Sandbox = Safety Bubble:** A sandbox creates an isolated container/environment. Anything that runs inside it:
    - Cannot touch the real file system
    - Cannot make permanent changes
    - Can be completely deleted/reset after testing — like it never happened
3. **Security Analyst Skill:** This is a core skill for SOC analysts, malware researchers, and incident responders — testing suspicious files before deciding if they're safe.

🧠 **What You'll Learn:**

- Sandboxing isolates applications from the real OS
- How to safely open a suspicious/test file (WikiWorm test file) inside the sandbox
- How the sandboxed app behaves differently than if runs normally
- How to clean up / delete all sandbox traces after testing

Machine to use: AdminMachine-1

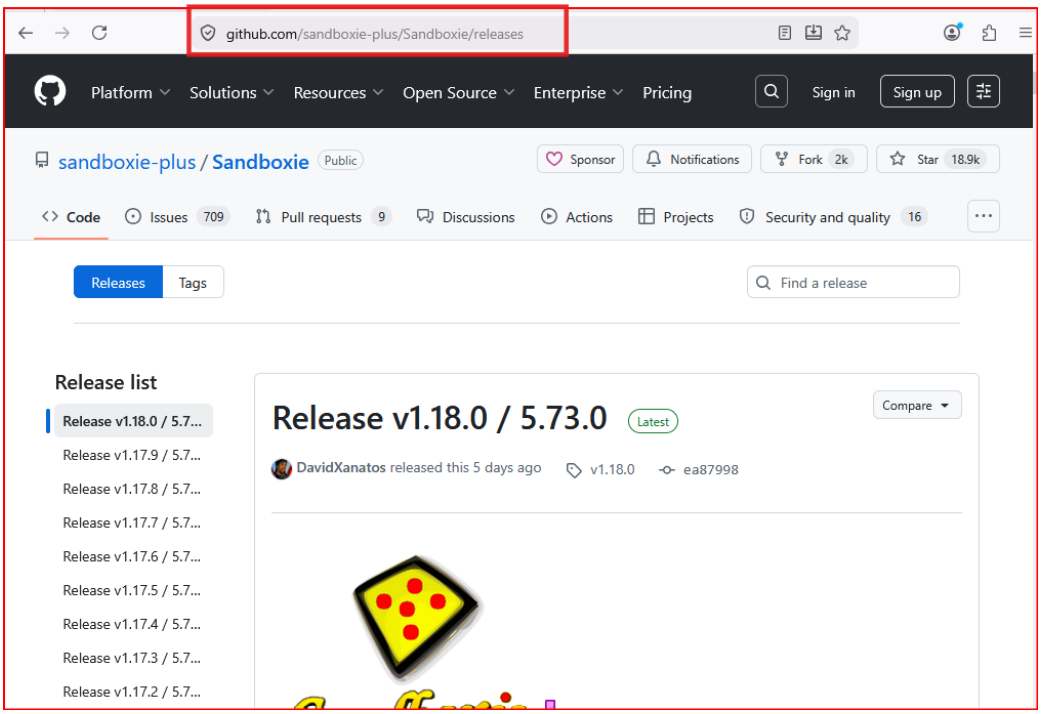
Download Sandboxie-Plus

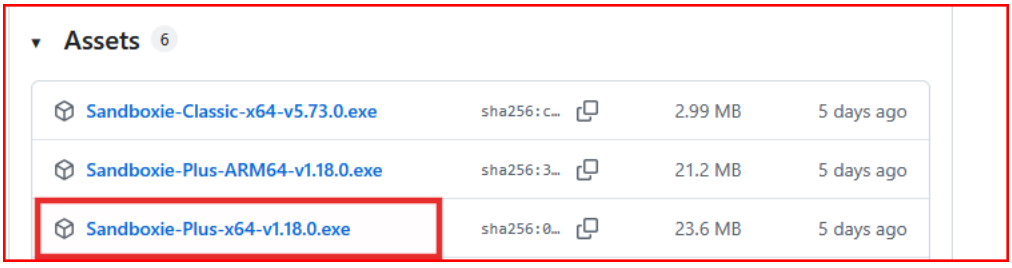
WHAT: Download the official Sandboxie-Plus installer.

WHY: We need the sandboxing tool itself before we can isolate and test the malware file. Using the official GitHub source ensures we're not accidentally downloading a fake/infected installer.

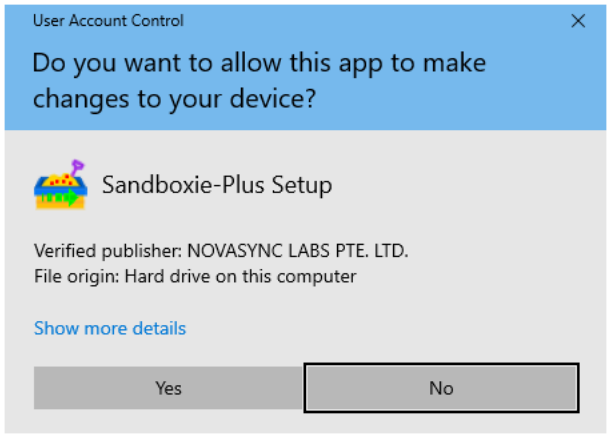
HOW:

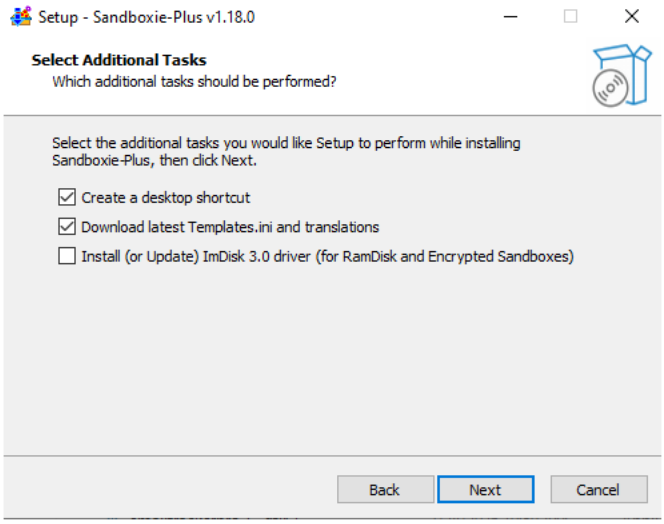
1. Open your browser on AdminMachine-1
2. Go to: https://github.com/sandboxie-plus/Sandboxie/releases
3. Find the latest release at the top (Scroll down)
4. Download the file ending in .exe (example: Sandboxie-Plus-x.xx.x-Windows-x64.exe)

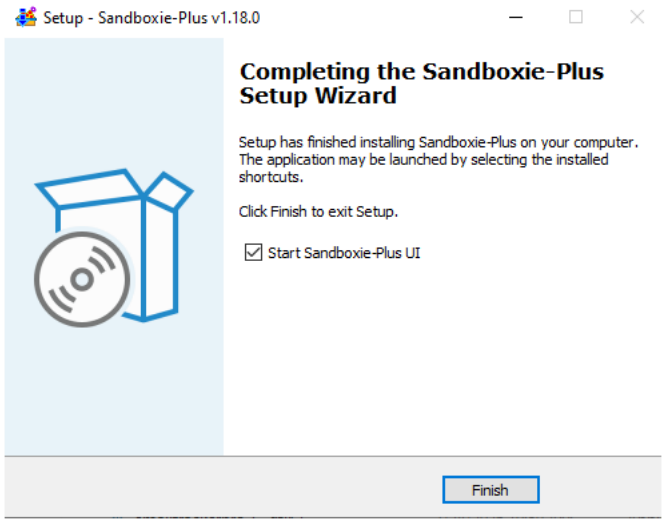




Run Sandboxie plus & Perform installation







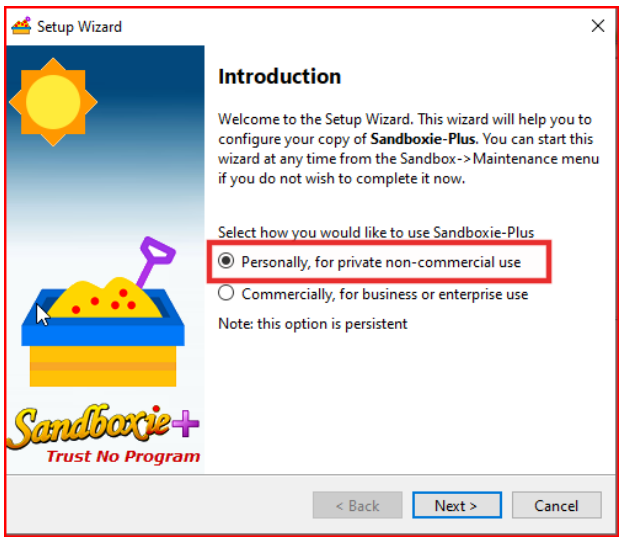
Usage Selection

Keep the 1st option selected (it already is):

- "Personally, for private non-commercial use" — SELECTED

WHY: This is a lab/educational setting, not business use.

1. Keep the first radio button checked
2. Click "Next >" button



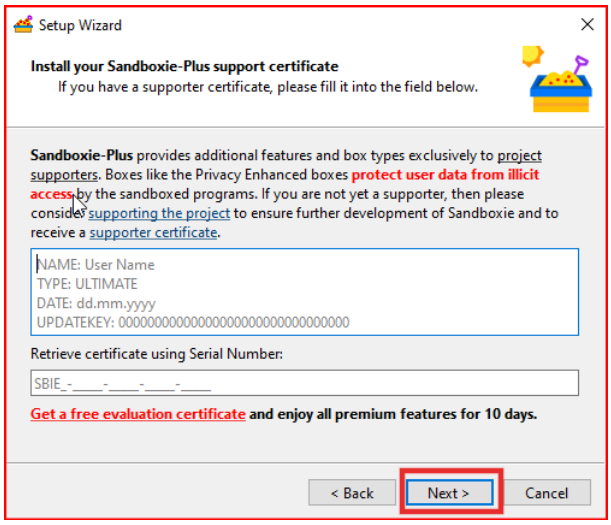
Support Certificate (Optional)

Leave the fields EMPTY — do NOT fill anything in.

WHY:

- The certificate is optional
- We only need basic Sandboxie-Plus functionality for this lab
- The free version is enough to test the malware file

1. Click "Next >" button

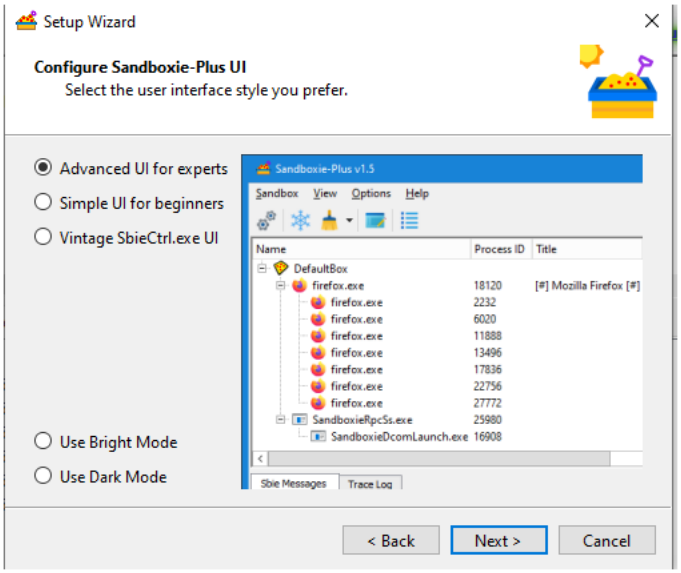


UI Configuration

Keep the 1st option selected (it already is):

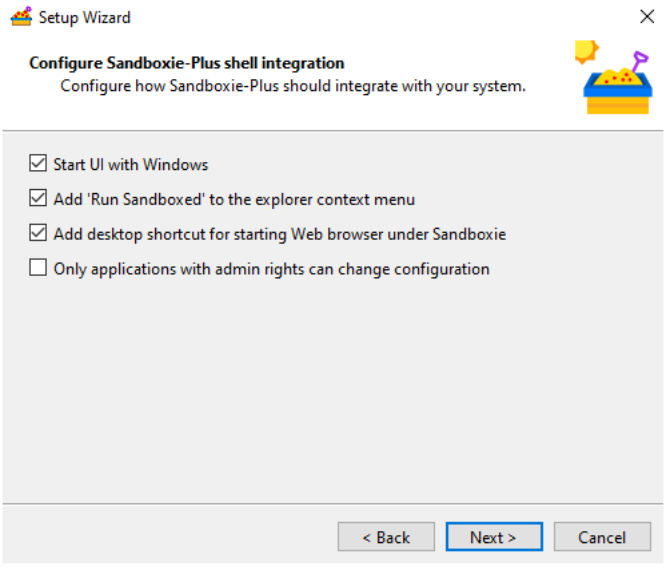
- "Advanced UI for experts" SELECTED

  WHY: This shows more details and controls — better for understanding the lab.



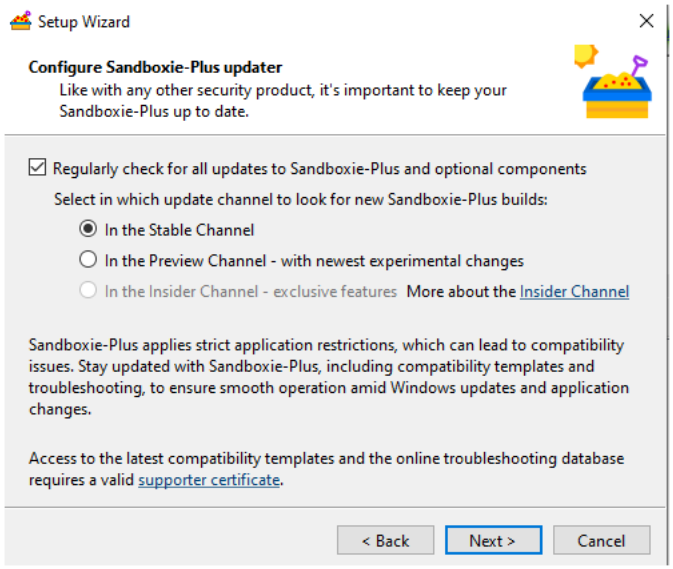
Shell Integration

Keep all checkboxes AS THEY ARE:



Updater Configuration

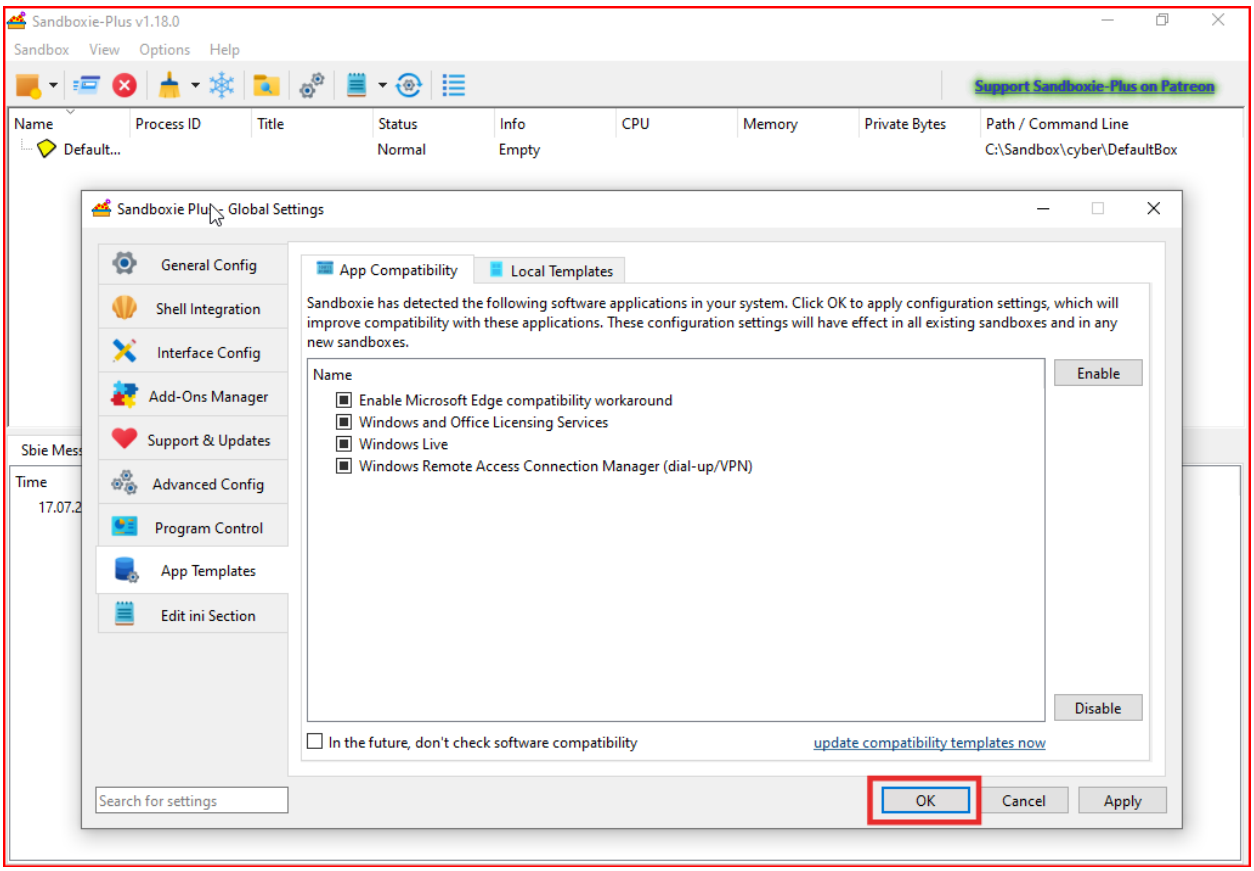
Keep all settings AS THEY ARE:



Installation Complete

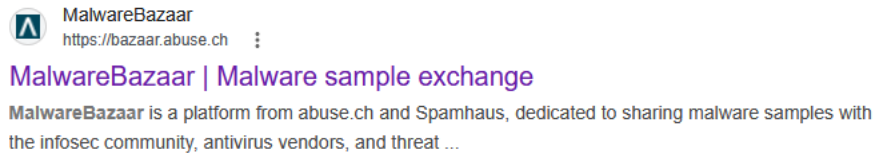
Sandboxie-Plus has successfully installed!

A "Global Settings" dialog appeared showing compatibility settings. This is automatic configuration — you don't need to change anything.

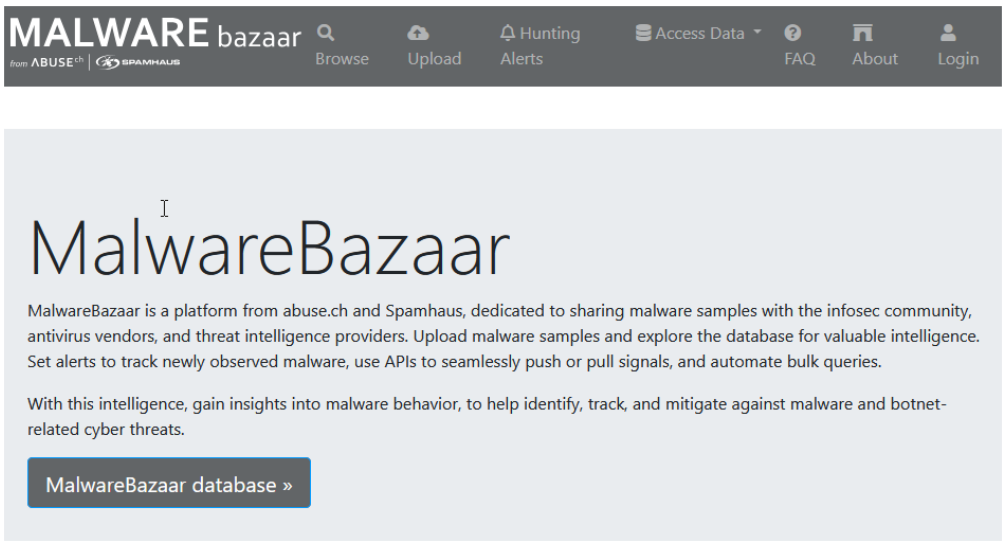


Malware Bazaar

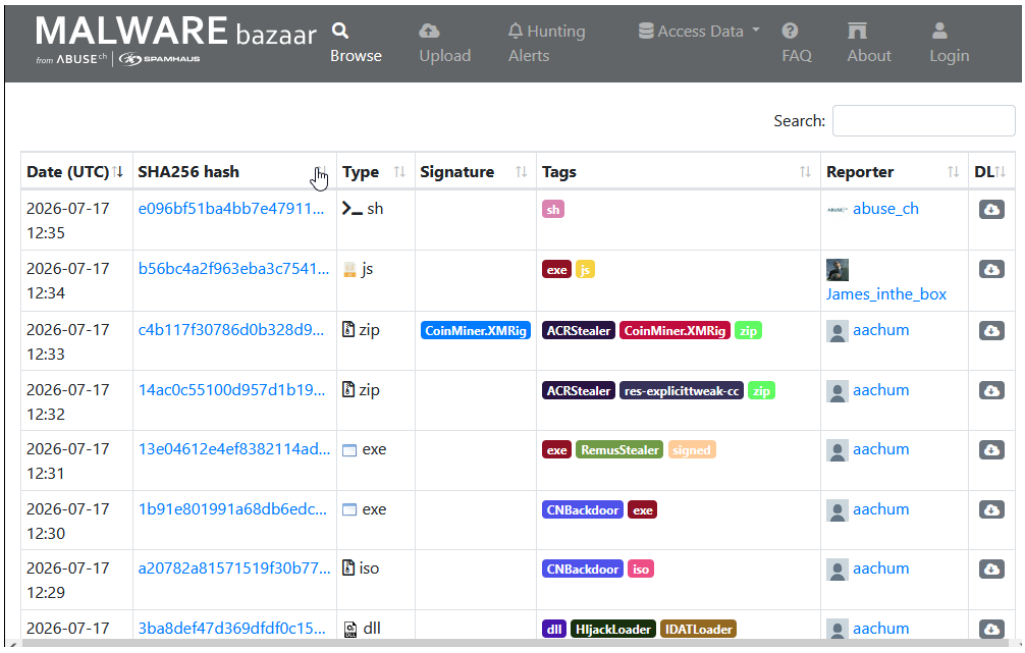
Now go to malware bazaar and download a malware to run in the sandbox



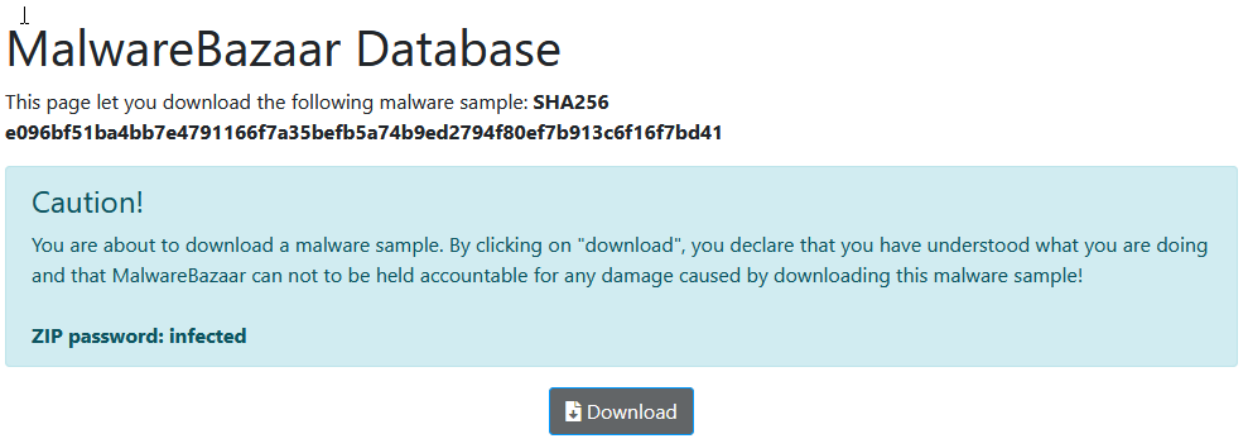
Now click on Database



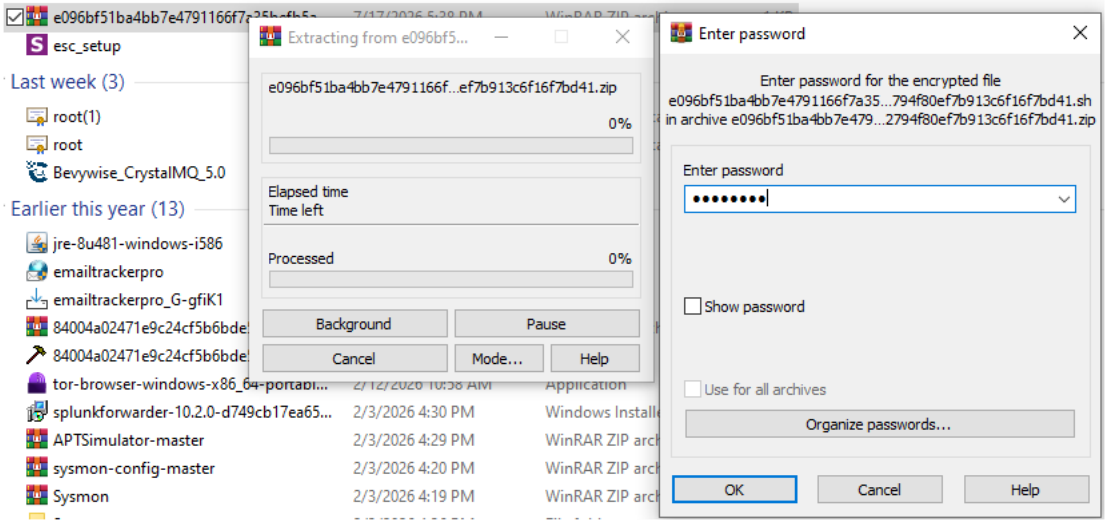
Select a malware to download



Download the malware



Extract the malware file and enter password "infected"



Confirm Malware File Location

1. Open File Explorer on AdminMachine-1
2. Navigate to where you download the malware file (usually Downloads folder)

⚠ CRITICAL SAFETY RULE:

- NEVER open/execute this file directly on the real system
- ONLY run it inside the Sandboxie sandbox (isolated environment)
- We will launch it from within Sandboxie in the next step

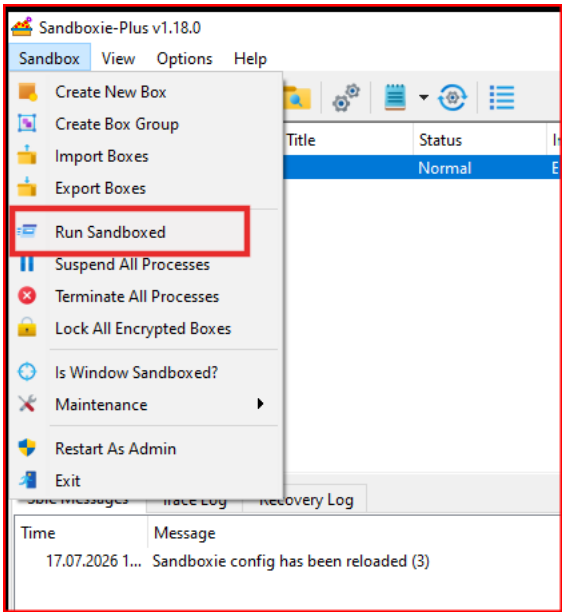
Run Malware File Inside Sandbox

WHAT YOU SEE: The Sandboxie menu is open with "Run Sandboxed" option visible.

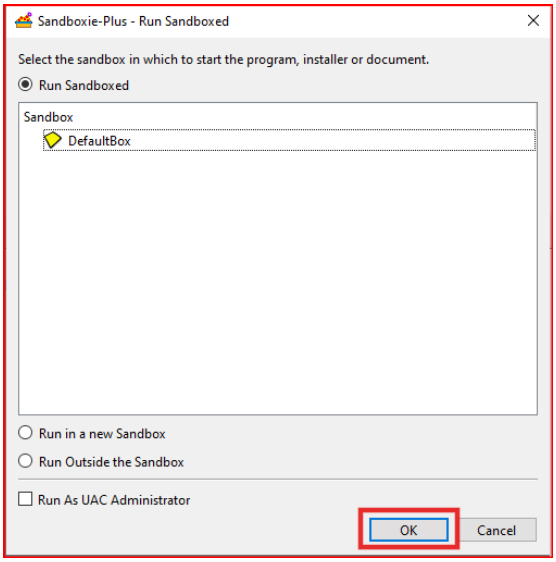
ACTION:

1. Click on "Run Sandboxed" (you can see it in the menu)

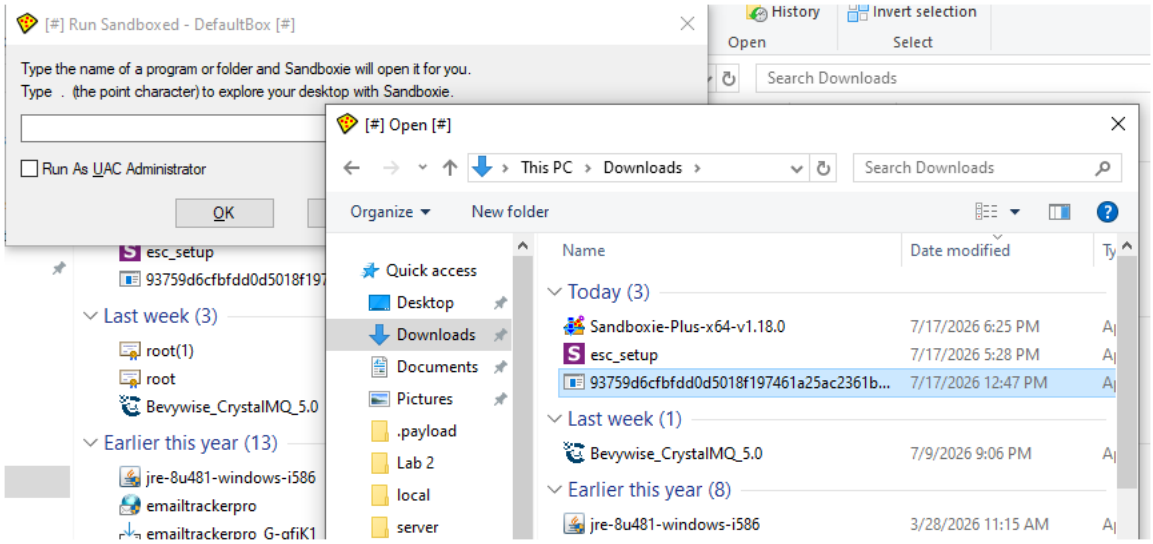
   WHAT HAPPENS: A file browser dialog will open where you can select your malware file.



Select Default sandbox and press OK



Select the malicious malware file (Browse & Select)

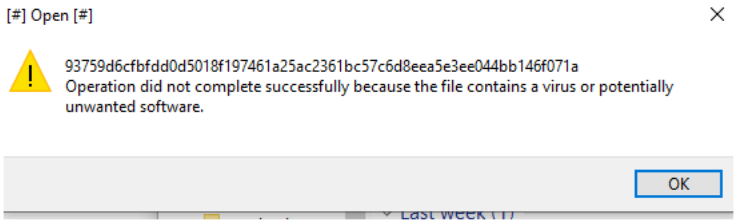


WHAT THIS MEANS:

The warning says: "Operation did not complete successfully because the file contains a virus or potentially unwanted software."

This proves:

- The file is genuinely malicious (Windows Defender detected it)
- Windows blocked it from running on the real system (good security)
- This is why we NEED sandboxing — to safely test it in isolation

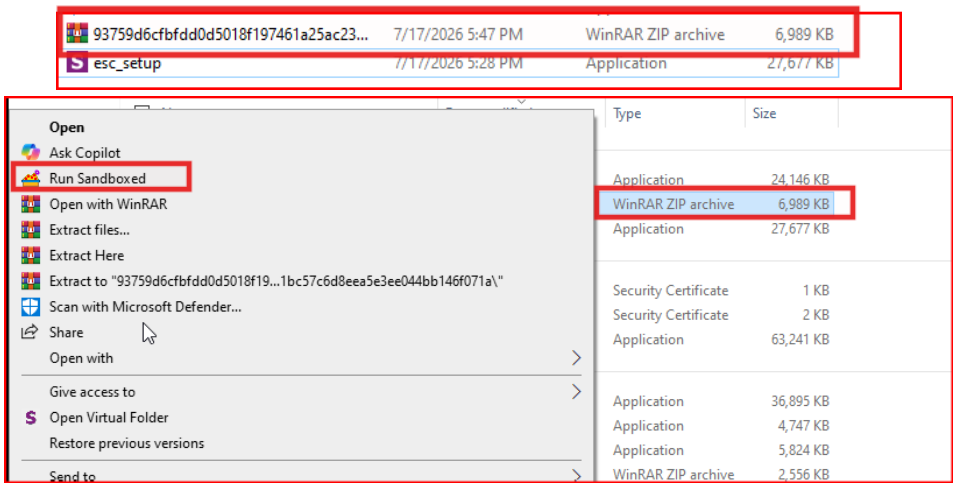


Close This Warning & Use Proper Method

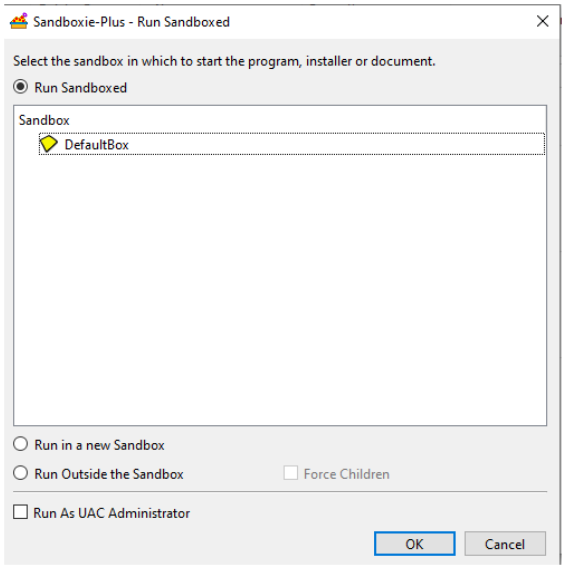
1. Click "OK" to close this warning dialog

Run Malware File Inside Sandbox (Correct Method)

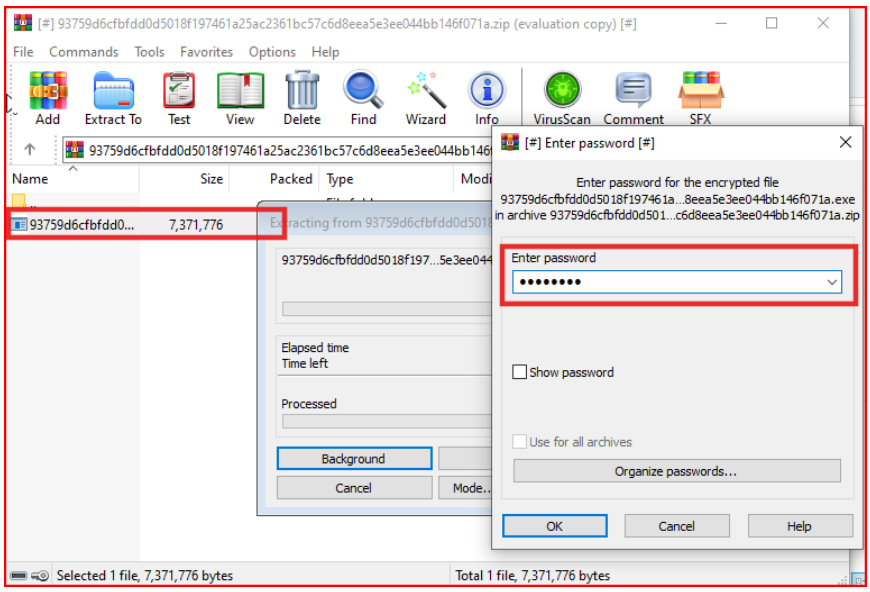
1. Open File Explorer on your desktop
2. Navigate to where your malware file is located (Downloads folder most likely)
3. Find the malware file (the one with hash name: 9375... etc)
4. Right-click on the malware file
5. From the context menu, select "Run Sandboxed" (this option was added during Sandboxie installation)



Select DefaultBox and press ok



Now in the sandbox you'll be asked to extract and run the file so double click on the file and enter the password "infected" to extract it

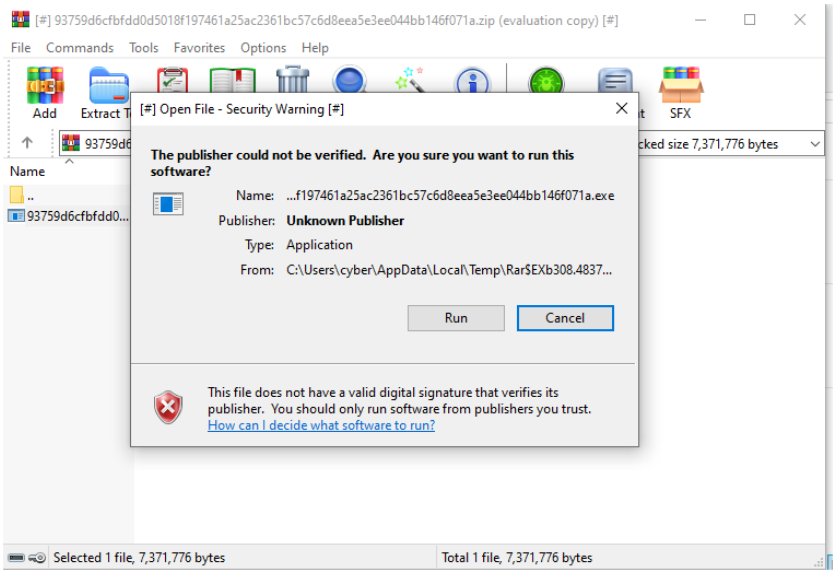


Click "Run" This is the Security Warning

WHAT THIS DIALOG SHOWS:

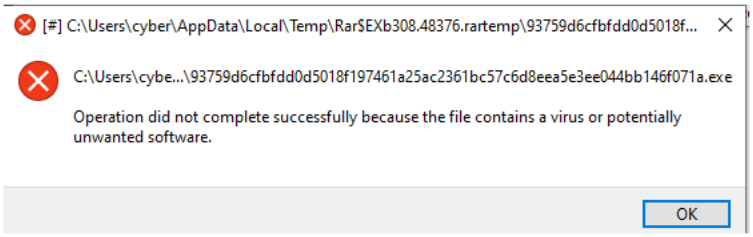
- Unknown Publisher (expected for malware)
- No valid digital signature (expected for malware)
- This is a legitimate security warning from Windows

IMPORTANT: The file is already set to run inside Sandboxie sandbox (DefaultBox) from our previous steps. So clicking "Run" will:



WHAT HAPPENED:

The malware file executed INSIDE the Sandboxie sandbox, and Windows Defender (running inside the sandbox) detected and blocked it:

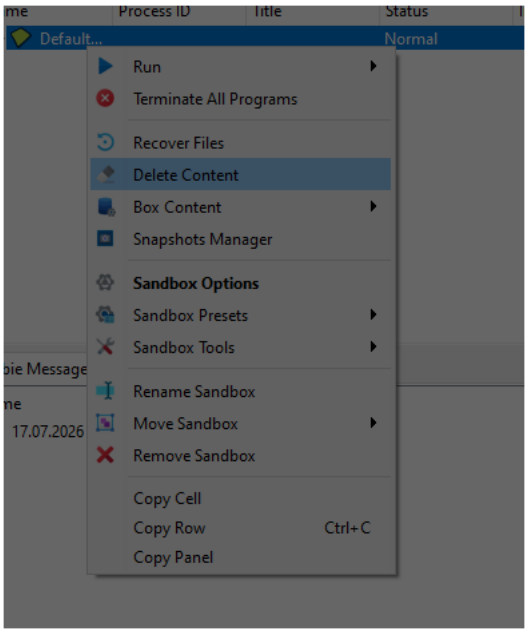


WHY THIS IS IMPORTANT:

- The malware ran, but only inside the isolated Sandboxie environment
- The real Windows system remained completely safe
- This is exactly what security analysts do: test suspicious files safely before deciding if they're threats

Cleanup Sandbox

1. Click "OK" to close this dialog
2. Go back to Sandboxie-Plus window (should still be open)
3. Right-click on "DefaultBox"
4. Select "Delete Contents" or "Delete Sandbox" to clean up the isolated environment



🎓 Key Learning Outcomes

What You Learned:

1. Sandboxing Concept: Creating isolated environments to safely test untrusted files
2. Malware Testing Safely: Running actual malware without risking the real OS
3. Defense in Depth: Layered security (sandbox + Windows Defender working together)
4. Incident Response Skill: How security analysts safely examine suspicious files
5. Cleanup Procedures: Proper removal of test artifacts after analysis

---

Conclusion

Module 9 successfully demonstrated the core principle underlying administrative application security: trust nothing by default, and verify controls actually work rather than assuming they do. Each lab moved beyond simple configuration into genuine validation — attempting real blocks, running real (contained) malware, and diagnosing real conflicts between overlapping security mechanisms.

The two necessary tool substitutions do not diminish the module's completeness; they reflect the reality of working in a security field where tools become deprecated or gated behind licensing, and where the underlying skill — understanding *how* and *why* a control works — matters more than any single vendor's product. In both substituted cases, the replacement tool was selected specifically because it preserved the original lab's security objective.